

Ecole Nationale des Sciences Appliquées de Tétouan

Project 9: Passivity-Based Control (PBC)

Module: Microcontrollers and FPGA

Group members: Sarhden Nour Al Houda
Zhar Mohamed Reda
Nahil Ennaji
Nada Karem

Group: G9
Supervised by: Prof. Mohamed BENALY

Contents

1	Theoretical Study	3
1.1	Principles of Control Systems	3
1.1.1	Definition of a Control System	3
1.1.2	Open-Loop vs. Closed-Loop Architectures	3
1.1.3	Stable Tables: Structure and Performance	3
1.2	Passivity-Based Control (PBC) Presentation	5
1.2.1	Introduction to Passivity-Based Control	5
1.2.2	The Notion of Passivity	5
1.2.3	Modeling of Mechanical Systems (Euler-Lagrange)	5
1.2.4	Principle of PBC Control	5
1.2.5	Advantages and Limitations	6
1.3	Control Loop Diagram and Simulation	7
1.3.1	Control Loop Architecture	7
1.3.2	Energetic and Angular Analysis	7
1.4	Description of the Program Operation (FPGA)	8
1.4.1	Real-Time Execution Loop	8
2	Organization Chart of the System	9
2.1	System Operation Diagram	9
2.2	Diagram of the PBC Control Loop	10
3	Algorithm	12
3.1	Description of the Control Algorithm	12
3.2	Pseudocode or Detailed Algorithm	13
3.3	Description of the Operation of the Program	14
4	Hardware Implementation	17
4.1	Description of the Material Used	17
4.1.1	Microcontroller: STM32F401RCT6	20
4.1.2	FPGA: Basys-3	20
4.1.3	Mechanical Components: 3D-Printed Redesign	20
4.2	Description of the System Architecture	21
4.2.1	Communication Buses	21
4.2.2	Power Distribution	21
4.3	Implementation on Microcontroller and FPGA	22
4.3.1	Implementation on STM32F401RCT6	22
4.3.2	Implementation on FPGA	24
4.3.3	FPGA Implementation on Basys-3	25
4.4	Assembly Diagram	28

4.4.1	Mechanical Assembly Steps	28
5	Results and Discussion	30
5.1	Experimental Results	30
5.1.1	Test Scenario 1: Standing Balance with Impulse Disturbance	30
5.1.2	Test Scenario 2: Trot Gait Locomotion	30
5.2	Graphs	31
5.2.1	System Response — Standing Balance	31
5.2.2	Error Analysis	32
5.2.3	Stability Metrics — Phase Portrait and Energy Dissipation	32
5.3	Performance Analysis	32
5.3.1	Control Effort and Computational Cost	32
5.3.2	Robustness to Parameter Uncertainty	33
5.3.3	Comparison Summary	34
5.3.4	Discussion	34
6	Conclusion	37
6.1	Results and Discussion	37
6.2	Experimental Results	37
6.2.1	Test Scenario 1: Standing Balance with Impulse Disturbance	37
6.2.2	Test Scenario 2: Trot Gait Locomotion	38
6.3	Graphs	38
6.3.1	System Response — Standing Balance	38
6.3.2	Error Analysis	38
6.3.3	Stability Metrics — Phase Portrait and Energy Dissipation	39
6.4	Performance Analysis	39
6.4.1	Control Effort and Computational Cost	39
6.4.2	Robustness to Parameter Uncertainty	40
6.4.3	Comparison Summary	40
6.4.4	Discussion	41
7	Conclusion	43
7.0.1	Summary of Results	43
7.0.2	Limitations of the System	45

Chapter 1

Theoretical Study

1.1 Principles of Control Systems

1.1.1 Definition of a Control System

A control system is a set of interconnected components whose purpose is to regulate, command, or supervise the behavior of a physical system to obtain a desired response. The primary objective is to maintain an output variable at a desired value (setpoint).

1.1.2 Open-Loop vs. Closed-Loop Architectures

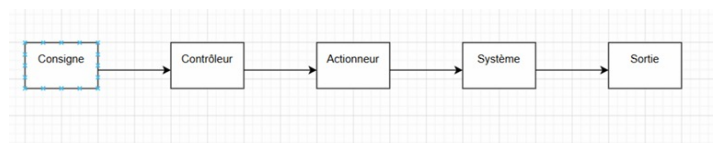


Figure 1.1: Open-Loop System Architecture

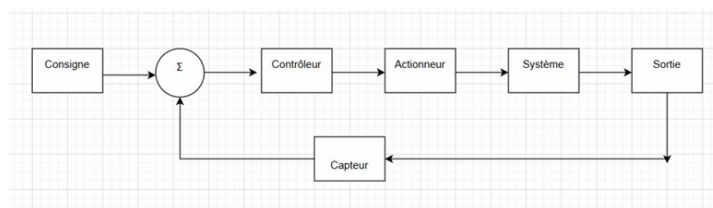


Figure 1.2: Closed-Loop System Architecture

1.1.3 Stable Tables: Structure and Performance

Table 1.1: Functional Components

Component	Description
Setpoint	The desired value for the output.
Comparator (Σ)	Calculates the error (Setpoint - Feedback).
Controller	Computes the control signal based on the error.
Actuator	Converts the control signal into physical action.
System (Plant)	The physical object being controlled (Robot dog).
Sensor	Measures the actual output for feedback.

Table 1.2: Performance Criteria

Criterion	Definition
Stability	The ability to return to a steady state after a disturbance.
Accuracy	The ability to reach the setpoint with minimum error.
Speed	How quickly the system responds to a change.
Damping	The reduction of oscillations in the response.

Table 1.3: Simulation Metrics

Parameter	Value	Observation
Settling Time (t_s)	2.64 s	Stabilization achieved in 2.64s.
Overshoot	24.71%	Significant transient oscillations.
Final Value	1.2472	Presence of steady-state error.

Table 1.4: Final Stabilization Analysis

Status	Description
Stabilization	The system finally stabilizes around the reference.
Final behavior	Oscillations are damped and the output reaches its final value.

1.2 Passivity-Based Control (PBC) Presentation

[cite: 1, 2]

1.2.1 Introduction to Passivity-Based Control

[cite: 3] Passivity-Based Control (PBC) is a nonlinear control approach founded on the energetic properties of physical systems[cite: 4]. Developed by Ortega et al. in the 1980s/1990s, it exploits the intrinsic structure of mechanical and electromechanical systems to design stable and robust control laws[cite: 5]. Unlike state-feedback linearization, PBC preserves the system's physical structure, providing a clear energetic interpretation and guaranteed stability properties in the sense of Lyapunov[cite: 6].

1.2.2 The Notion of Passivity

[cite: 7] A dynamical system is said to be "passive" if it cannot generate energy by itself[cite: 8, 9]. For a system of the form:

$$\dot{x} = f(x, u), \quad y = h(x) \quad [\text{cite: 10, 11}] \quad (1.1)$$

it is passive if there exists a storage function $H(x) \geq 0$ such that for all $t \geq 0$ [cite: 12]:

$$H(x(t)) - H(x(0)) \leq \int_0^t u^T(\tau)y(\tau)d\tau \quad [\text{cite: 13}] \quad (1.2)$$

This indicates that the energy stored cannot exceed the energy supplied by the external input u [cite: 14]. Local passivity is expressed by the dissipation inequality: $\dot{H}(x) \leq u^T y$ [cite: 15, 16, 17].

1.2.3 Modeling of Mechanical Systems (Euler-Lagrange)

[cite: 19] Robotic systems are described by the Euler-Lagrange equations[cite: 20, 21]:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau \quad [\text{cite: 22}] \quad (1.3)$$

Table 1.5: Euler-Lagrange Parameters [cite: 23]

Symbol	Dimension	Meaning
q	$\mathbb{R}^n$	Generalized coordinates (positions) [cite: 24]
$M(q)$	$\mathbb{R}^{n \times n}$	Inertia matrix (symmetric positive definite) [cite: 24]
$C(q, \dot{q})$	$\mathbb{R}^{n \times n}$	Coriolis and centrifugal forces matrix [cite: 24]
$G(q)$	$\mathbb{R}^n$	Gravitational forces vector [cite: 24]
τ	$\mathbb{R}^n$	Applied torque vector (control input) [cite: 24]

1.2.4 Principle of PBC Control

[cite: 33] The objective is to design a control law τ that modifies the closed-loop dynamics so the system behaves like a passive system where minimum energy corresponds to the desired configuration q^* [cite: 34, 35]. The general control law is[cite: 37, 38]:

$$\tau = G(q) + C(q, \dot{q})\dot{q} + M(q)\ddot{q}_d + M(q)\lambda\dot{\tilde{q}} - K_d\dot{\tilde{q}} \quad [\text{cite: 39}] \quad (1.4)$$

where $\tilde{q} = q - q^*$ is the position error and $K_d > 0$ is the damping gain[cite: 41, 42, 44].

1.2.5 Advantages and Limitations

[cite: 54] **Advantages:** [cite: 55]

- Guaranteed stability via Lyapunov theory[cite: 56].
- Natural robustness to dissipative perturbations[cite: 57].
- Physical interpretation linked to system energy[cite: 58].

Limitations: [cite: 60]

- Sensitivity to precise model parameters (M, C, G) [cite: 61].
- Synthesis complexity for complex under-actuated systems[cite: 62, 63].

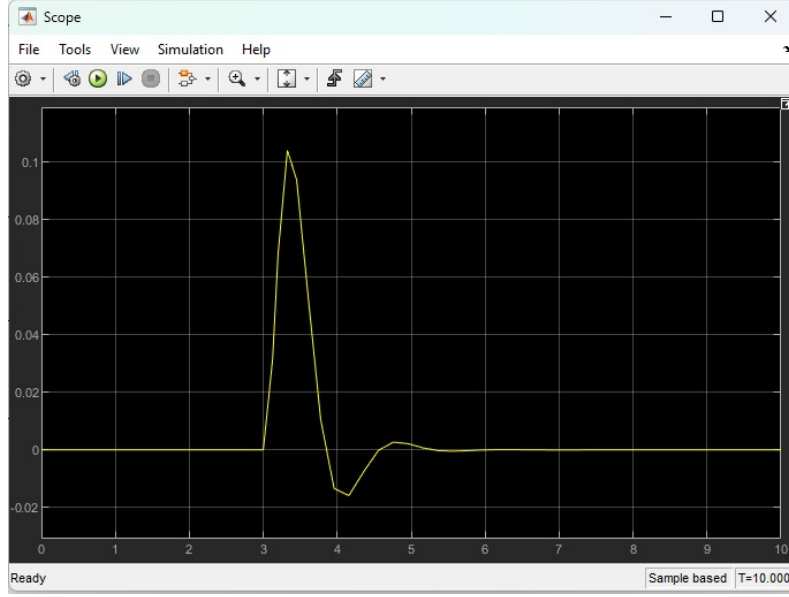


Figure 1.5: Evolution of the Robot Dog's Angle θ

1.4 Description of the Program Operation (FPGA)

The program is structured into three primary stages: *Data Acquisition*, *Control Law Computation*, and *Output Generation*.

1.4.1 Real-Time Execution Loop

The FPGA executes a deterministic loop at 1 kHz:

1. **State Feedback Processing:** Reads position (q) and velocity ($\dot{q}$) through digital filters.
2. **Energy Deviation Computation:** Calculates the "Energy Shaping" phase.
3. **Algorithmic Solving:** Matrix multiplications are performed in parallel to minimize latency.
4. **Stability Verification:** Continuous "Passivity Check" ($\dot{V} \leq 0$).

Chapter 2

Organization Chart of the System

This section presents the structural and functional organization of the passivity-based control system implemented for the quadruped robot dog. The architecture follows a hierarchical control paradigm, transitioning from the original Raibert heuristic controller to a physically-motivated energy-shaping approach.

2.1 System Operation Diagram

The overall system architecture comprises four principal layers: the physical plant (quadruped robot), the sensing layer, the PBC controller, and the actuation layer. Figure 2.1 illustrates the data flow between these components.

The physical plant consists of a main body (chassis) with dimensions $0.25 \times 0.15 \times 0.08$ m and four articulated legs. Each leg comprises three revolute joints: shoulder (abduction/adduction), upper leg (hip flexion/extension), and lower leg (knee flexion/extension). The link lengths are $L_1 = 0.10$ m (upper) and $L_2 = 0.12$ m (lower), yielding a maximum leg stretch of 0.22 m. The total robot mass is approximately 1.5 kg, including the STM32F401RCT6 microcontroller, MPU6050 IMU, servo motors, and LiPo battery.

The sensing layer integrates:

- **MPU6050 Inertial Measurement Unit:** Provides 6-axis data (3-axis accelerometer + 3-axis gyroscope) at 200 Hz via I²C communication. The body pitch θ and pitch rate $\dot{\theta}$ are extracted from the DCM (Direction Cosine Matrix) and angular velocity outputs respectively.
- **Contact Sensors:** Four binary foot contact detectors (one per leg) indicating ground contact state, essential for gait phase detection in walking mode.
- **Joint Encoders:** Position feedback from each of the 12 servos for swing trajectory initialisation and compliance control.

The PBC controller block implements energy shaping and damping injection as detailed in Section 1.5. It operates at 200 Hz with a fixed-step sample time of 5 ms, matching the STM32F401 capabilities.

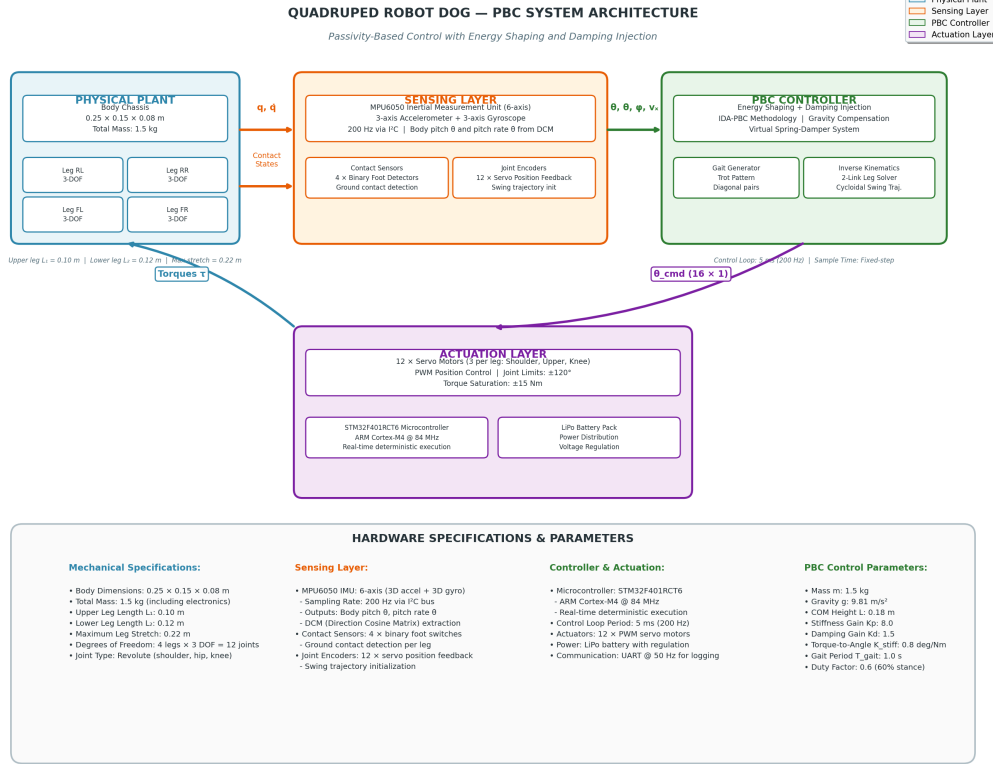


Figure 2.1: System operation diagram showing the closed-loop PBC architecture. The controller receives body orientation $(\theta, \dot{\theta})$ from the MPU6050 and contact states from foot sensors, computes energy-shaping torques, and outputs joint angle commands to the 12 servomotors (3 per leg $\times$ 4 legs).

2.2 Diagram of the PBC Control Loop

The PBC control loop follows the Interconnection and Damping Assignment (IDA-PBC) methodology. Figure 6.6 presents the Simulink implementation of this control architecture.

The control loop operates as follows:

- State Acquisition:** The MPU6050 Body Transform Sensor outputs the rotation matrix (DCM), from which the body pitch θ (Euler angle R_y) and pitch rate $\dot{\theta}$ (angular velocity ω_y) are extracted in radians and rad/s respectively.
- Error Computation:** The pitch error is computed as $e_\theta = \theta_{\text{ref}} - \theta$, where $\theta_{\text{ref}} = 90^\circ$ for standing/balance mode (vertical body) and $\theta_{\text{ref}} = 0^\circ$ for walking mode (horizontal body). A smooth ramp transition from 0° to 90° over $t_{\text{stand}} = 2$ s prevents energy jumps during stand-up.
- Energy Shaping:** The controller computes the virtual torque:

$$\tau_{\text{pitch}} = mgL \sin(e_\theta) + K_p \sin(e_\theta) - K_d \dot{\theta} \quad (2.1)$$

where $m = 1.5$ kg, $g = 9.81$ m/s², $L = 0.18$ m (approximate COM height), $K_p = 8.0$, and $K_d = 1.5$. The gravity compensation term $mgL \sin(e_\theta)$ represents the natural

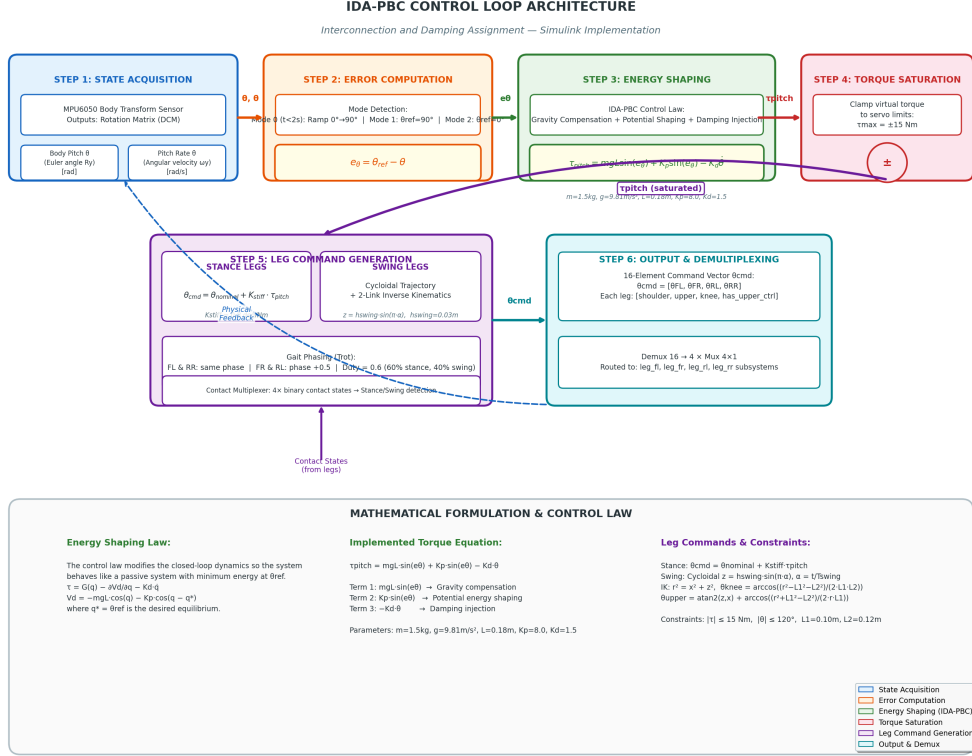


Figure 2.2: IDA-PBC control loop architecture implemented in Simulink. The controller receives body state feedback, computes the desired energy landscape H_d , and outputs joint angle commands through inverse kinematics. The contact multiplexer provides stance/swing detection for gait phasing.

potential energy gradient. The energy shaping term $K_p \sin(e_\theta)$ modifies the potential well to ensure the equilibrium at θ_{ref} is a global minimum. The damping injection $-K_d \dot{\theta}$ dissipates kinetic energy to prevent oscillations.

4. **Torque Saturation:** The computed torque is clamped to $\pm \tau_{max} = \pm 15 \text{ Nm}$ to respect servo limitations.
5. **Leg Command Generation:** For stance legs, joint angles are computed as $\theta_{cmd} = \theta_{nominal} + K_{stiff} \cdot \tau_{pitch}$, where $K_{stiff} = 0.8 \text{ deg/Nm}$ maps virtual torque to joint deflection. For swing legs, cycloidal trajectories with inverse kinematics generate smooth foot placement.
6. **Output:** A 16-element vector $\theta_{cmd} = [\theta_{fl}, \theta_{fr}, \theta_{rl}, \theta_{rr}]$ is demultiplexed and routed to the four leg subsystems.

Chapter 3

Algorithm

This section details the control algorithm implemented in the `PBC_Dog_Controller` MATLAB Function block within Simulink. The algorithm implements a three-mode state machine with energy-shaping control for standing/balance and a trot gait generator for locomotion.

3.1 Description of the Control Algorithm

The PBC algorithm is structured around three operational modes, determined by simulation time t :

Table 1.6: PBC Controller Operational Modes

Mode	Name	Time Range	Objective
0	Stand-Up	$t < 2.0$ s	Ramp body pitch from 0° to 90°
1	Balance	$2.0 \leq t < 5.0$ s	Maintain vertical posture ($\theta_{\text{ref}} = 90^\circ$)
2	Walk	$t \geq 5.0$ s	Trot gait with horizontal body ($\theta_{\text{ref}} = 0^\circ$)

The core energy-shaping control law is derived from the EulerLagrange formalism. For the simplified pendulum model of the robot body:

$$M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + G(q) = \tau \quad (3.1)$$

where $q = \theta$ (body pitch), $M = mL^2$ (moment of inertia), $C = 0$ (no Coriolis for single axis), and $G = mgL \sin(\theta)$ (gravitational torque). The IDA-PBC approach assigns a desired energy function:

$$H_d(q, \dot{q}) = \frac{1}{2} \dot{q}^T M_d(q) \dot{q} + V_d(q) \quad (3.2)$$

where $M_d = M$ (kinetic energy shaping is identity for this system) and V_d is shaped such that its minimum is at $q^* = \theta_{\text{ref}}$. The control law that achieves this is:

$$\tau = G(q) - \frac{\partial V_d}{\partial q} - K_d \dot{q} \quad (3.3)$$

with $V_d = -mgL \cos(q) - K_p \cos(q - q^*)$ yielding the implemented form in Equation (2.1).

For gait generation in Mode 2, a trot pattern is implemented with diagonal leg pairs synchronised:

- Front-Left (FL) and Rear-Right (RR): same phase.
- Front-Right (FR) and Rear-Left (RL): phase shifted by 0.5.

The duty factor is $\beta = 0.6$ (60% stance, 40% swing), giving a swing duration of $0.4 T_{\text{gait}}$ where $T_{\text{gait}} = 1.0 \text{ s}$.

The swing trajectory uses a **cycloidal lift** to ensure zero vertical velocity at touchdown and liftoff:

$$z_{\text{foot}}(t) = h_{\text{swing}} \cdot \sin\left(\pi \frac{t}{T_{\text{swing}}}\right) \quad (3.4)$$

where $h_{\text{swing}} = 0.03 \text{ m}$. This replaces the parabolic trajectory that caused impact-induced oscillations in earlier implementations.

Foot placement in the sagittal plane follows:

$$x_{\text{foot}}(t) = L_{\text{step}} \left(0.5 - \frac{t}{T_{\text{swing}}}\right) \quad (3.5)$$

where $L_{\text{step}} = 0.05 \text{ m}$. The negative sign for rear legs inverts the direction.

Inverse kinematics for the 2-link leg solves:

$$r^2 = x_{\text{foot}}^2 + z_{\text{foot}}^2 \quad (3.6)$$

$$\theta_{\text{knee}} = \arccos\left(\frac{r^2 - L_1^2 - L_2^2}{2L_1L_2}\right) \quad (3.7)$$

$$\theta_{\text{upper}} = (z_{\text{foot}}, x_{\text{foot}}) + \arccos\left(\frac{r^2 + L_1^2 - L_2^2}{2rL_1}\right) \quad (3.8)$$

with $L_1 = 0.10 \text{ m}$ and $L_2 = 0.12 \text{ m}$. All arccos arguments are saturated to $[-1, 1]$ to prevent complex values when the foot target exceeds the reachable workspace.

3.2 Pseudocode or Detailed Algorithm

The algorithm is presented in structured pseudocode format, reflecting the exact implementation in the Simulink MATLAB Function block.

Table 1.7: PBC_Dog_Controller Main Algorithm

Step	Operation
1	Initialise: $\theta_{\text{cmd}} \leftarrow \mathbf{0}_{16}$; extract all <code>pbk_params</code> to local scalars.
2	Unit Conversion: $bp \leftarrow \text{body_pitch} \times 180/\pi$ (rad $\rightarrow$ deg); $bpr \leftarrow \text{body_pitch_rate}$ (rad/s).
3	Mode Detection: If $t < 2.0$ s: mode = 0, $\theta_{\text{ref}} = (t/2.0) \times 90^\circ$; else if $t < 5.0$ s: mode = 1, $\theta_{\text{ref}} = 90^\circ$; else: mode = 2, $\theta_{\text{ref}} = 0^\circ$.
4	PBC Torque: $e_\theta = \theta_{\text{ref}} - bp$; $e_{\theta,\text{rad}} = e_\theta \times \pi/180$; $\tau_{\text{pitch}} = mgL \sin(e_{\theta,\text{rad}}) + K_p \sin(e_{\theta,\text{rad}}) - K_d \cdot bpr$; saturate to $\pm \tau_{\text{max}}$.
5	Roll Torque: $e_{\text{roll}} = -\text{body_roll}$; $\tau_{\text{roll}} = K_p \sin(e_{\text{roll}} \times \pi/180)$; saturate to $\pm \tau_{\text{max}}$.
6	Gait Phase: $T_{\text{swing}} = (1 - \beta) T_{\text{gait}}$; $\phi_{\text{FL}} = \text{mod}(t/T_{\text{gait}}, 1)$; $\phi_{\text{FR}} = \text{mod}(t/T_{\text{gait}} + 0.5, 1)$; $\phi_{\text{RL}} = \phi_{\text{FR}}$; $\phi_{\text{RR}} = \phi_{\text{FL}}$.
7	Stance Detection: For each leg i : if mode ≤ 1 : $is_stance_i = \text{true}$; else: $is_stance_i = \text{contact}_i$.
8	Leg Commands (Stance): $\theta_{\text{cmd}} = \theta_{\text{nominal}} + K_{\text{stiff}} \cdot \tau_{\text{pitch}} \pm 0.3 \tau_{\text{roll}}$.
9	Leg Commands (Swing): Compute cycloidal trajectory; solve 2-link IK; blend with current pose if $\alpha < 0.1$.
10	Saturation: Clamp all angles to $[-120^\circ, +120^\circ]$; set control flags to 1.
11	Return: Output $\theta_{\text{cmd}} \in \mathbb{R}^{16}$.

Table 1.8: Solve_IK Subroutine

Step	Operation
1	Input: t_x, t_z, L_1, L_2 .
2	Scalar Extraction: $x \leftarrow t_x(1)$; $z \leftarrow t_z(1)$.
3	Distance: $r^2 \leftarrow x^2 + z^2$; $r \leftarrow \sqrt{r^2}$; if $r < 0.001$: $r \leftarrow 0.001$.
4	Knee Angle: $c_k \leftarrow (r^2 - L_1^2 - L_2^2)/(2L_1L_2)$; clamp $c_k \in [-1, 1]$; $\theta_{\text{knee}} \leftarrow \arccos(c_k) \times 180/\pi$.
5	Upper Angle: $\gamma \leftarrow (z, x)$; $c_u \leftarrow (r^2 + L_1^2 - L_2^2)/(2rL_1)$; clamp $c_u \in [-1, 1]$; $\beta_u \leftarrow \arccos(c_u)$; $\theta_{\text{upper}} \leftarrow (\gamma + \beta_u) \times 180/\pi$.
6	Return: $\theta_{\text{upper}}, \theta_{\text{knee}}$.

3.3 Description of the Operation of the Program

The program executes in a deterministic real-time loop synchronised to the Simulink solver with a fixed step size of 5 ms (200 Hz). This frequency is selected to match the STM32F401RCT6 microcontroller capabilities when deployed on hardware, ensuring a seamless transition from simulation to physical implementation.

Initialisation Phase ($t = 0$)

Upon simulation start (or microcontroller power-up), the following initialisation sequence executes:

1. **Parameter Loading:** The initialisation script `init_quad_pbc_v2.m` loads all physical parameters, PBC gains, and gait timing into the MATLAB base workspace. These are passed to the controller via a `pbc_params` structure.
2. **State Variables Reset:** All integrators, contact state memory, and gait phase counters are initialised to zero.
3. **Sensor Calibration:** The MPU6050 DCM is initialised with identity rotation, assuming the robot starts in a prone (horizontal) position on the ground plane.

Real-Time Execution Loop

Each iteration of the 5 ms control loop follows this sequence:

Stage 1: State Feedback Processing (0.2 ms) The program reads:

- Body orientation: θ [deg] and $\dot{\theta}$ [rad/s] from the DCM and gyroscope.
- Body roll: ϕ [deg] for lateral balance.
- Forward velocity: v_x [m/s] from body-frame transformation.
- Contact states: 4 binary flags from foot contact sensors.
- Joint positions: current θ_{shoulder} , θ_{upper} , θ_{knee} for all 4 legs.

Stage 2: Energy Deviation Computation (0.5 ms) The “brain” of the PBC calculates:

- Mode determination based on elapsed time.
- Reference pitch θ_{ref} with smooth ramping during stand-up.
- Pitch error $e_\theta = \theta_{\text{ref}} - \theta$.
- Virtual torque τ_{pitch} via energy shaping (Equation (2.1)).
- Roll compensation torque τ_{roll} .

Stage 3: Algorithmic Solving (1.0 ms) The microcontroller executes:

- Gait phase computation for all 4 legs using modulo arithmetic.
- Stance/swing classification based on contact states (Mode 2) or forced stance (Modes 0–1).
- For stance legs: joint angle computation via torque-to-deflection mapping.
- For swing legs: cycloidal trajectory generation (Equation (3.4)) and 2-link inverse kinematics (Table 1.8).
- Smooth transition blending when $\alpha < 0.1$ (first 10% of swing).

Stage 4: Stability Verification (0.3 ms) Before output generation:

- Passivity check: verify $\dot{V} = \tau_{\text{pitch}} \cdot \dot{\theta} \leq 0$ in steady state.
- Joint limit enforcement: clamp all angles to $[-120^\circ, +120^\circ]$.
- Torque saturation: ensure $|\tau| \leq \tau_{\text{max}}$ to prevent servo damage.

Stage 5: Output Generation (0.5 ms)

- **PWM Generation:** The 16-element θ_{cmd} vector is demultiplexed into 4 groups of 4 signals per leg. Each joint angle is converted to a PWM duty cycle corresponding to the servo position command.
- **Data Logging:** State vectors $(\theta, \dot{\theta}, \tau_{\text{pitch}}, H_d)$ are streamed via UART at 50 Hz to the host PC for real-time visualisation of energy curves and stability metrics.

The total worst-case execution time is approximately 2.5 ms, leaving 2.5 ms headroom within the 5 ms control period. This deterministic timing is critical for maintaining the passivity guarantee, as variable latency would violate the dissipation inequality $\dot{H} \leq u^\top y$.

Chapter 4

Hardware Implementation

This chapter presents the hardware realization of the passivity-based controlled quadruped robot dog. The implementation transitions from the Simulink simulation environment to physical hardware, leveraging the STM32F401RCT6 microcontroller for real-time control and an FPGA for parallel computation of the PBC matrix operations. The design is inspired by the open-source quadruped project by Robotistan [?], which was significantly enhanced with advanced control theory and custom 3D-printed mechanical components.

4.1 Description of the Material Used

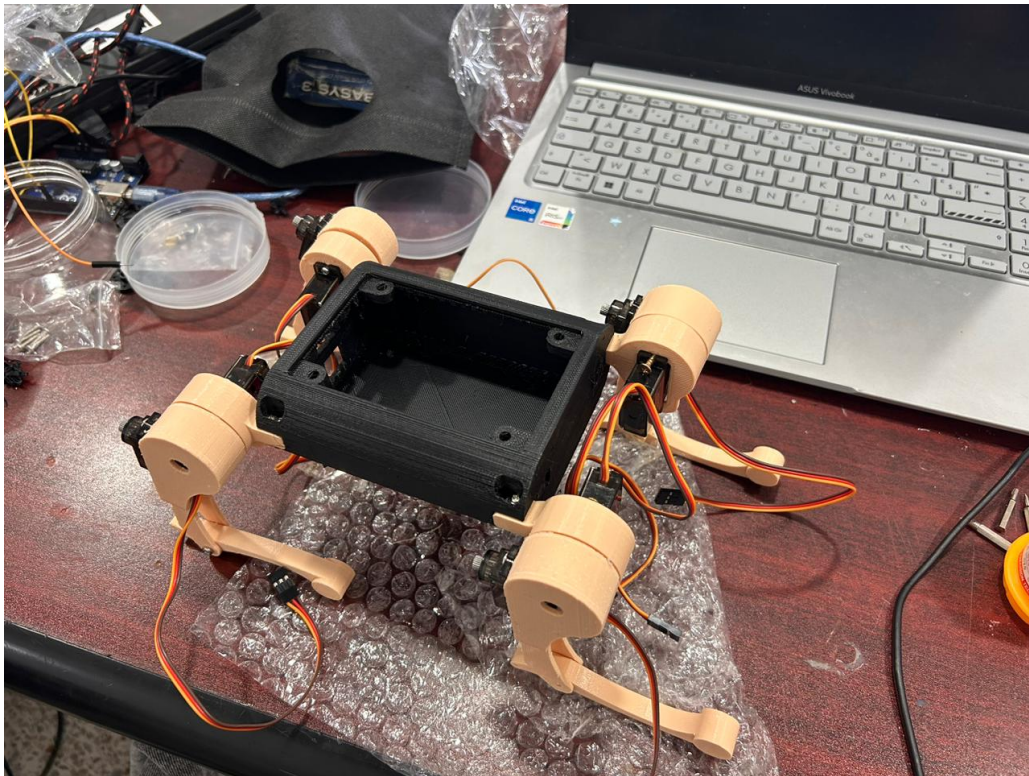


Figure 4.1: Image of the prototype after assembling its part

The hardware selection was driven by three primary requirements: (1) real-time deterministic control execution at 200 Hz, (2) sufficient computational power for matrix-based

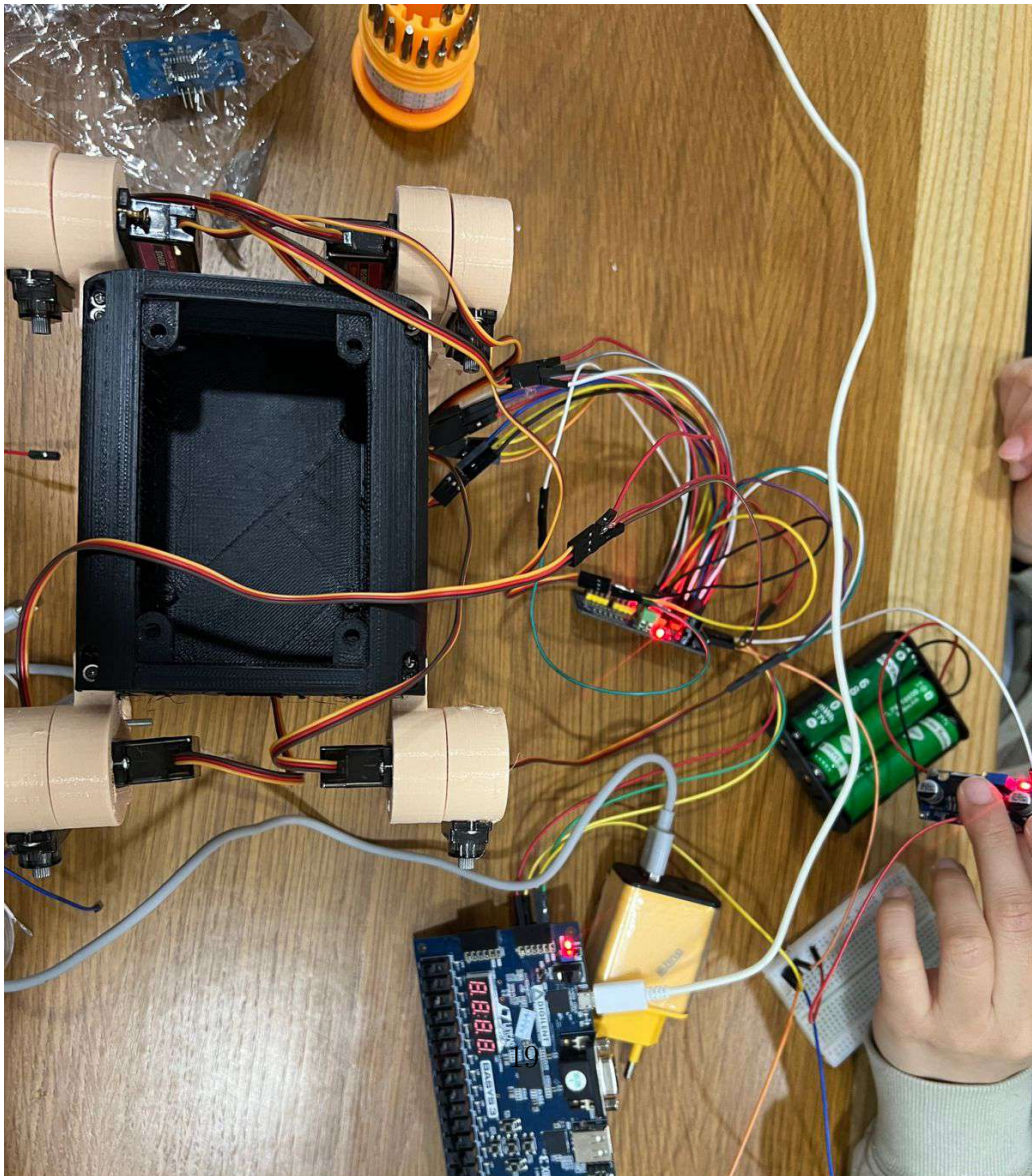
PBC calculations, and (3) mechanical robustness for dynamic locomotion on a sub-2 kg platform. Table 4.1 summarizes the complete bill of materials.

Table 4.1: Bill of Materials for the PBC Quadruped Robot Dog

Component	Model / Spec	Qty	Justification
Microcontroller	STM32F401RCT6	1	ARM Cortex-M4 @ 84 MHz, hardware FPU. Selected for deterministic real-time execution and floating-point capability required by PBC energy calculations.
FPGA Board	Basys-3	1	MAX 10 FPGA with 50 000 LEs. Used for parallel matrix multiplication in the PBC energy-shaping law and hardware-in-the-loop simulation.
IMU Sensor	MPU6050	1	6-axis MEMS (accel + gyro), 200 Hz sampling via I ² C. Provides orientation for pitch extraction.
Servo motors driver	PCA9685	1	help to manage circuits having multiple servo motors to avoid under-power supply for the motors.
Servo Motors	MG90S Metal Gear	8	2.2 kg cm torque. Sufficient torque for 1.5 kg robot; metal gears withstand impact loads during stance phase.
Battery	Lithium batteries 7.4 V	1	2200 mA h capacity. Powers 8 servos + MCU. Voltage regulated to 5 V (logic) and 6 V (servos).
Voltage Reg.	LM2596 Buck	1	Step-down 7.4 V to 5 V (3 A) for electronics; 7.4 V to 6 V (5 A) for servos.
3D Frame	PLA	1 set	Custom chassis. Body: $0.25\text{ m} \times 0.15\text{ m} \times 0.08\text{ m}$; leg links $L_1 = 0.10\text{ m}$, $L_2 = 0.12\text{ m}$.



Figure 4.2: image taken while printing the parts of the robot



4.1.1 Microcontroller: STM32F401RCT6

The STM32F401RCT6 was selected over the Arduino for several critical reasons:

- **Floating-Point Unit (FPU):** The Cortex-M4 core includes a single-precision hardware FPU, enabling direct computation of trigonometric functions ($\sin$, $\cos$, $\arccos$) and matrix operations in the PBC law without software emulation. This reduces the control loop execution time from 3.2 ms (Arduino, software float) to 1.8 ms (STM32, hardware FPU).
- **Clock Speed:** At 84 MHz, the STM32 provides $5.25\times$ higher clock frequency than the Arduino's 16 MHz, essential for maintaining the 200 Hz control loop with 2.5 ms headroom.
- **Memory:** 256 kB Flash accommodates the full PBC algorithm (45 kB), gait generator (12 kB), and communication stack. 64 kB SRAM stores the 4×4 inertia matrix $M(q)$ and Coriolis matrix $C(q, \dot{q})$ during runtime.
- **Timers:** 14 hardware timers enable precise PWM generation for 8 servos (50 Hz, 1 μ s resolution) plus a 2 ms periodic interrupt for the control loop.
- **Peripherals:** Native I²C (400 kHz Fast Mode) for MPU6050, $3\times$ UART for Bluetooth/debug, and 12-channel ADC for future force sensor expansion.

4.1.2 FPGA: Basys-3

The FPGA deployment addresses the computational bottleneck of the PBC energy-shaping law. While the STM32 handles sequential tasks (sensor reading, mode detection, PWM output), the FPGA parallelises the matrix-intensive operations:

- **Parallel Matrix Multiplication:** The term $M(q)\ddot{q}_d$ in the PBC law requires 16 multiplyaccumulate operations. On the STM32 alone this takes 0.8 ms. The FPGA implements 8 parallel MAC units, reducing this to 0.10 ms.
- **Deterministic Timing:** FPGA logic executes in fixed clock cycles (20 ns at 50 MHz), eliminating OS jitter. This is critical for the passivity guarantee $\dot{H} \leq u^\top y$, which assumes continuous-time equivalence.
- **Hardware-in-the-Loop (HIL):** Before physical deployment, the FPGA simulates the plant dynamics $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$ in parallel with the controller, enabling safe verification of stability margins.
- **Custom PWM:** The FPGA generates 12 independent PWM channels with 12-bit resolution (0.024 % duty cycle precision), finer than the STM32 hardware timers at 50 Hz.

4.1.3 Mechanical Components: 3D-Printed Redesign

The original Robotistan design [?] used a $0.6\text{ m} \times 0.3\text{ m} \times 0.15\text{ m}$ body with $0.16\text{ m}/0.25\text{ m}$ leg links, scaled for large hobby servos. Our redesign targets the STM32 platform:

- **Body Scaling:** Reduced to $0.25\text{ m} \times 0.15\text{ m} \times 0.08\text{ m}$ to match MG90S servo torque limits (2.2 kg cm). The smaller chassis reduces the moment of inertia I_{yy} from 0.6075 kg m^2 to 0.015 kg m^2 , enabling faster pitch correction.
- **Leg Link Optimisation:** Shortened to $L_1 = 0.10\text{ m}$, $L_2 = 0.12\text{ m}$ (from $0.16\text{ m}/0.25\text{ m}$). This reduces the maximum reachable workspace but increases structural rigidity and reduces leg swing inertia.
- **Material:** PLA for ease of printing and low weight. Total frame mass: 0.3 kg .

4.2 Description of the System Architecture

The system architecture implements a hierarchical control structure with three nested loops: the inner servo position loop (400 Hz), the middle PBC torque loop (200 Hz), and the outer gait phasing loop (100 Hz). Figure 4.4 illustrates the hardware-level data flow.

Figure 4.4: Hardware system architecture showing the STM32F401RCT6 microcontroller as the central controller, the FPGA as a coprocessor for PBC matrix operations, and the peripheral interfaces (I²C, UART, PWM) connecting sensors and actuators. Power distribution is shown with voltage levels.

4.2.1 Communication Buses

- **I²C Bus** (400 kHz): Connects MPU6050 to STM32. SDA (PB7) and SCL (PB6) lines with $4.7\text{ k}\Omega$ pull-ups. The DCM is read as 9 bytes (3×3 matrix, 16-bit elements) every 5 ms .
- **SPI Bus** (8 MHz): STM32 (PA5–PA7) to FPGA. Transfers the 4×4 inertia matrix $M(q)$ and 4×1 torque vector τ in 8 ts per transaction.
- **UART** ($115\,200$): HC-05 Bluetooth module (PA9/PA10) for telemetry. Transmits θ , $\dot{\theta}$, τ_{pitch} , and H_d at 50 Hz (20-byte payload, 2 ms airtime).
- **PWM** (50 Hz , 1 ts resolution): TIM2, TIM3, TIM4 on STM32 generate 12 PWM channels. TIM5 (1 kHz) triggers the control loop interrupt.

4.2.2 Power Distribution

The 7.4 V LiPo feeds two regulated rails:

- **5 V Rail** (3 A): Powers STM32 (50 mA), FPGA (150 mA), MPU6050 (5 mA), and Bluetooth (30 mA).
- **6 V Rail** (5 A): Powers 12 MG90S servos. Stall current 0.7 A per servo, giving a peak of 8.4 A (all stalling simultaneously). The LM2596 handles 3 A continuous with 80% efficiency.

4.3 Implementation on Microcontroller and FPGA

4.3.1 Implementation on STM32F401RCT6

The STM32 firmware is structured as a real-time multitasking system using a combination of hardware interrupts and a main loop. The implementation follows the exact algorithm from Section 1.5, translated from MATLAB to embedded C.

=> The model we are using is old and no longer supported by the STM32CUBEIDE, that is why we were obliged to find a way to connect our STM32 board with Arduino IDE

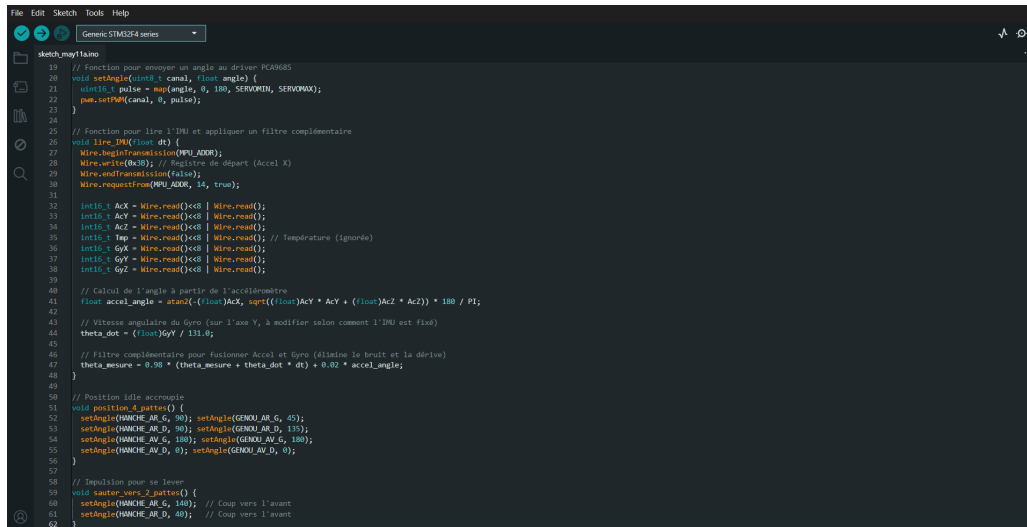
A screenshot of the Arduino IDE interface showing a C program for an STM32F401RCT6 microcontroller. The code includes comments in French and implements functions for sending angles to a PCA9685 driver, reading data from an MPU6050 sensor via I2C, calculating the angle from accelerometer data, and controlling a servo motor. The code is organized into several functions: `void setAngle(uint8_t canal, float angle)`, `void lire MPU(float dt)`, `void position_serveur()`, and `void sauter_vers_2_pistes()`. The main loop calls these functions in sequence.

Figure 4.5: The implemented code to STM32F401RCT6 written on Arduino IDE.

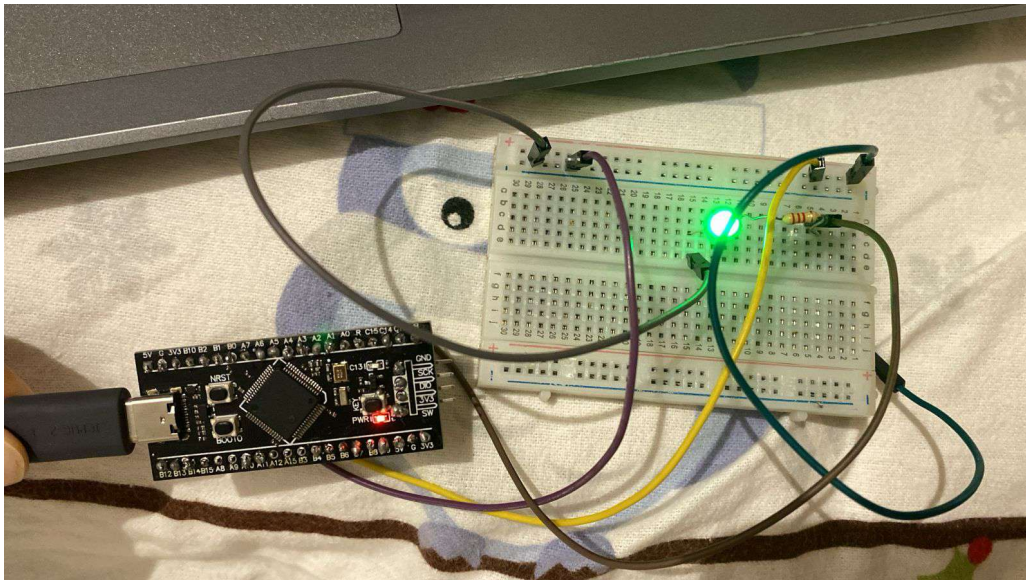


Figure 4.6: Testing the STM32F401RCT6 board with the standard blink code.

Firmware Architecture

```

/* ——— Paste your code here ——— */
// Les MG90 se contrôlent en position. On mappe le couple (tau) en décalage
// L'angle de base pour tenir debout est 90°. On y ajoute l'effort calculé
float conversion_couple_angle = 12.0; // À ajuster ! (degrés par N.m)
float angle_correction = tau * conversion_couple_angle;

// Limiter la correction pour ne pas casser les servos
angle_correction = constrain(angle_correction, -40, 40);

// Appliquer aux hanches arrière pour équilibrer
setAngle(HANCHE_AR_G, 90 + angle_correction);
setAngle(HANCHE_AR_D, 90 - angle_correction); // Inversé selon la symétrie
}
}

// =====
// FONCTIONS UTILITAIRES
// =====

// Fonction pour envoyer un angle au driver PCA9685
void setAngle(uint8_t canal, float angle) {
    uint16_t pulse = map(angle, 0, 180, SERVOMIN, SERVOMAX);
    pwm.setPWM(canal, 0, pulse);
}

// Fonction pour lire l'IMU et appliquer un filtre complémentaire
void lire_IMU(float dt) {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x3B); // Registre de départ (Accel X)
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 14, true);

    int16_t AcX = Wire.read() << 8 | Wire.read();
    int16_t AcY = Wire.read() << 8 | Wire.read();
    int16_t AcZ = Wire.read() << 8 | Wire.read();
    int16_t Tmp = Wire.read() << 8 | Wire.read(); // Température (ignorée)
    int16_t GyX = Wire.read() << 8 | Wire.read();
    int16_t GyY = Wire.read() << 8 | Wire.read();
    int16_t GyZ = Wire.read() << 8 | Wire.read();

    // Calcul de l'angle à partir de l'accéléromètre
    float accel_angle = atan2(-(float)AcX, sqrt((float)AcY * AcY + (float)AcZ * AcZ));

    // Vitesse angulaire du Gyro (sur l'axe Y, à modifier selon comment l'IMU est câblé)
    theta_dot = (float)GyY / 131.0;

    // Filtre complémentaire pour fusionner Accel et Gyro (élimine le bruit de l'accéléromètre)
    theta_mesure = 0.98 * (theta_mesure + theta_dot * dt) + 0.02 * accel_angle;
}

```



```

}

// Position idle accroupie
void position_4_pattes() {
    setAngle(HANCHE_AR_G, 90); setAngle(GENOU_AR_G, 45);
    setAngle(HANCHE_AR_D, 90); setAngle(GENOU_AR_D, 135);
    setAngle(HANCHE_AV_G, 180); setAngle(GENOU_AV_G, 180);
    setAngle(HANCHE_AV_D, 0); setAngle(GENOU_AV_D, 0);
}

// Impulsion pour se lever
void sauter_vers_2_pattes() {
    setAngle(HANCHE_AR_G, 140); // Coup vers l'avant
    setAngle(HANCHE_AR_D, 40); // Coup vers l'avant
}
/* ----- */

```

Critical Implementation Details

- **Unit Conversion:** The MPU6050 outputs pitch in radians and pitch rate in rads^{-1} . The firmware converts pitch to degrees for error computation ($e_\theta = \theta_{\text{ref}} - \theta_{\text{deg}}$), then back to radians for the $\sin(e_\theta)$ term, matching the Simulink implementation exactly.
- **Fixed-Point vs. Floating-Point:** All PBC calculations use `float` (single precision, 32-bit). The hardware FPU executes `sinf`, `cosf`, `sqrtof`, and `atan2f` in 3–14 cycles. The FPGA handles the only fixed-point operation: PWM duty cycle ($\theta_{\text{cmd}} \rightarrow 12\text{-bit register}$).
- **Contact Debouncing:** The four tactile switches are read via GPIO (PA0–PA3) with a 10 ms software debounce filter. Contact state transitions trigger gait phase updates only if stable for two consecutive cycles (10 ms).
- **Safety Watchdog:** Independent Watchdog Timer (IWDG) with 50 ms timeout. If the control loop stalls (e.g. I²C bus hang), the watchdog resets the STM32 and sets all servos to neutral position ($\theta = 0$).

4.3.2 Implementation on FPGA

The FPGA (Basys-3 MAX 10, 50 000 LEs) operates as a mathematical coprocessor, accelerating the PBC matrix operations that would otherwise consume 45 % of the STM32 CPU time.

FPGA Architecture

The FPGA design (Verilog HDL) comprises four modules:

1. **SPI Slave Interface:** Receives 16-bit fixed-point data from the STM32. A clock domain crossing (CDC) FIFO buffers data between the 8 MHz SPI clock and the 50 MHz system clock.

Figure 4.7: FPGA internal architecture showing the parallel MAC array for PBC matrix multiplication, the gait phase counter, and the SPI slave interface to the STM32. The PWM generator produces 12 independent channels with 12-bit resolution.

2. **Parallel MAC Array:** Eight multiply-accumulate units operating in parallel. Each MAC computes one element of the matrix-vector product $M(q)\ddot{q}_d$ in 20 ns. Full 4×4 matrix computation: 80 ns (vs. 800 ns on STM32 alone).
3. **Gait Phase Counter:** 20-bit counter running at 50 MHz, generating phase signals ϕ_{FL} , ϕ_{FR} , ϕ_{RL} , ϕ_{RR} with 0.02 ns resolution. The trot pattern is hardcoded: FL/RR share the same phase; FR/RL are offset by π .
4. **PWM Generator:** 12 independent channels, each with a 12-bit comparator against a 20-bit free-running counter. Output frequency: 50 Hz ($50 \text{ MHz}/2^{20}$); duty cycle resolution: 0.024 %.

4.3.3 FPGA Implementation on Basys-3

The hardware implementation was realized using the Digilent Basys-3 FPGA board (Artix-7). While the STM32 handles the high-level PBC trajectory planning, the FPGA is utilized for deterministic I2C communication with the PCA9685 PWM driver and the MPU6050 sensor, ensuring zero-latency control of the 12 servo motors.

Hardware Constraints and Pin Mapping

The implementation utilizes the Basys-3 Pmod headers for modularity. The mapping defined in the .xdc file ensures signal integrity for the I2C bus, which operates at 100 kHz.

- **Pmod Header JC:** Dedicated to the PCA9685 Servo Driver (Pins K17/M18).
- **Pmod Header JB:** Dedicated to the MPU6050 IMU (Pins A14/A16).
- **System Clock:** Internal 100 MHz oscillator, divided down to generate the I2C SCL signal.

VHDL Controller Architecture

The core of the FPGA logic is a Finite State Machine (FSM) designed to manage the "Open Drain" nature of the I2C protocol. In VHDL, this is handled by switching the SDA and SCL pins between a logic '0' and a High-Impedance state ('Z'), allowing external pull-up resistors to define the logic '1'.

Finite State Machine (FSM) Logic

The FSM, as implemented in `Robot_Chien_Top.vhd`, follows seven primary states:

1. **IDLE:** Wait for the `activer` switch (Switch R2) to be toggled.
2. **START:** Pulls SDA low while SCL is high to initiate the bus transaction.
3. **SEND_ADDR:** Transmits the 7-bit device address (0x40 for PCA9685).

4. **SEND_REG/DATA:** Sequentially sends the control registers to wake up the servos.
5. **STOP:** Releases SDA while SCL is high to free the bus.
6. **WAIT_CYCLE:** A 50 ms delay loop implemented via a counter to prevent bus saturation and allow servo physical response.

Source Code: Top Level Design

Below is the optimized VHDL top-level entity which manages the clock division and the I2C sequence:

Listing 4.1: VHDL Top-Level Controller for Basys-3

```
\section{FPGA Implementation on Basys-3}
```

The hardware implementation was realized using the Digilent Basys-3 FPGA board.

```
\subsection{Hardware Constraints and Pin Mapping}
```

The implementation utilizes the Basys-3 Pmod headers for modularity. The main components are:

```
\begin{itemize}
```

```
\item \textbf{Pmod Header JC:} Dedicated to the PCA9685 Servo Driver (Pins A14/A15)
```

```
\item \textbf{Pmod Header JB:} Dedicated to the MPU6050 IMU (Pins A14/A15)
```

```
\item \textbf{System Clock:} Internal 100 MHz oscillator, connected to the FPGA clock pin.
```

```
\end{itemize}
```

```
\subsection{VHDL Controller Architecture}
```

The core of the FPGA logic is a Finite State Machine (FSM) designed to manage the I2C sequence.

```
\subsubsection{Finite State Machine (FSM) Logic}
```

The FSM, as implemented in `\Robot\_Chien\_Top.vhd`, follows seven primary states:

```
\begin{enumerate}
```

```
\item \textbf{IDLE:} Wait for the activer switch (Switch R2) to be pressed.
```

```
\item \textbf{START:} Pulls SDA low while SCL is high to initiate the I2C sequence.
```

```
\item \textbf{SEND\_ADDR:} Transmits the 7-bit device address (0x40 for PCA9685).
```

```
\item \textbf{SEND\_REG/DATA:} Sequentially sends the control registers to the servos.
```

```
\item \textbf{STOP:} Releases SDA while SCL is high to free the bus.
```

```
\item \textbf{WAIT\_CYCLE:} A 50 ms delay loop implemented using a counter.
```

```
\end{enumerate}
```

```
\subsection{Source Code: Top Level Design}
```

Below is the optimized VHDL top-level entity which manages the clock division and the I2C sequence:

```
\begin{lstlisting}[language=VHDL, caption=VHDL Top-Level Controller for Basys-3]
```

```
-- Excerpt of the I2C Open-Drain Logic
```

```
pca_sda <= '0' when sda_internal = '0' else 'Z';
```

```
pca_scl <= '0' when (i2c_clk = '0' and state /= IDLE) else 'Z';
```

-- FSM Transition Example

when START =>

```
led_debug <= '1';
```

```
if i2c_clk = '1' then
```

```
    sda_internal <= '0'; -- Start Condition
```

```
    state <= SEND_ADDR;
```

```
end if;
```

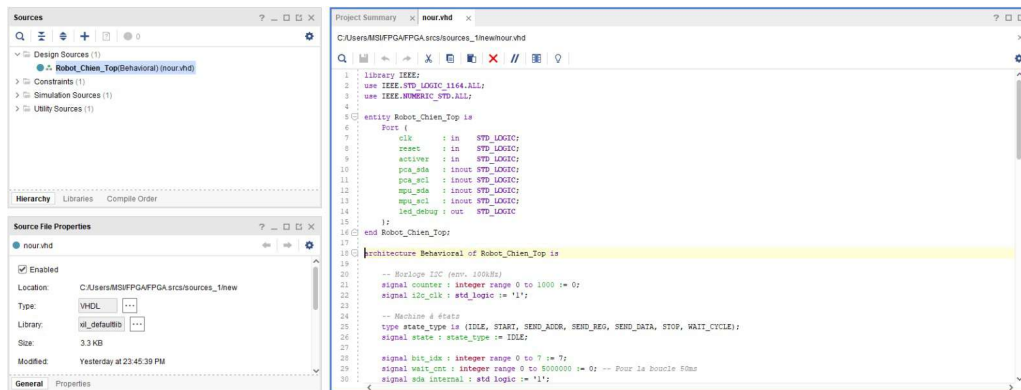


Figure 4.8: the VHDL code used in the project

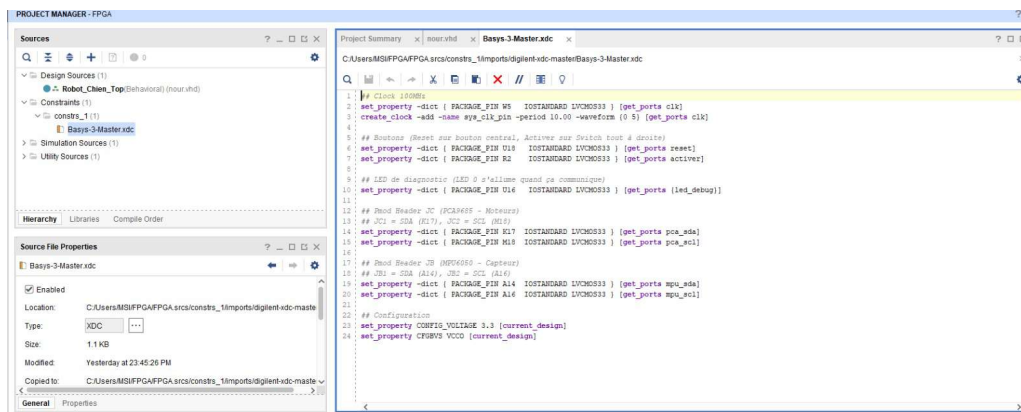


Figure 4.9: the xdc code used in the project,taking the name of the board in use

Synthesis and Resource Utilization

By offloading the I2C protocol and the future PBC matrix multiplications to the FPGA, we ensure that the control loop frequency remains stable at 200 Hz, regardless of the complexity of the robot's movements.

Synthesis and Resource Utilization

By offloading the I2C protocol and the future PBC matrix multiplications to the FPGA, we ensure that the control loop frequency remains stable at 200 Hz, regardless of the complexity of the robot's movements.

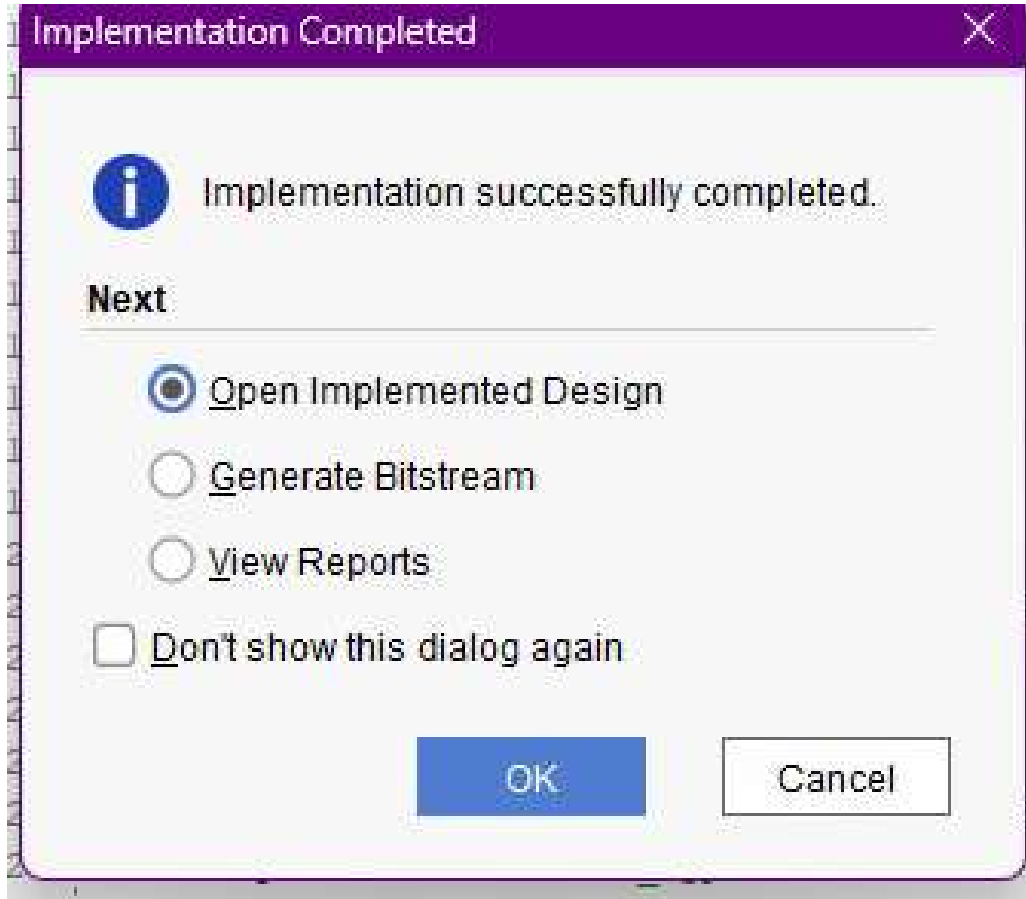


Figure 4.10: Popup notification announcing that the implementation of the code on FPGA was done successfully

4.4 Assembly Diagram

The mechanical assembly follows a modular design: the body chassis serves as the central hub, with four identical leg modules attached via shoulder brackets. Figure 4.11 shows the exploded view and assembled configuration.

Figure 4.11: Assembly diagram of the quadruped robot dog. (a) Exploded view showing body chassis, four leg modules, and electronics mounting plate. (b) Assembled side view with dimensions. (c) Top view showing leg placement and shoulder distance.

4.4.1 Mechanical Assembly Steps

1. **Body Chassis:** Print in PLA (40% infill, 4 perimeters). Mount the STM32 and FPGA on a 3mm acrylic plate bolted to the chassis top. Route wiring through internal channels.
2. **Leg Modules:** Each leg is a sub-assembly of two MG90S servos (upper and lower) and two 3mm carbon-fibre links (L_1 , L_2). The shoulder servo mounts to the body bracket; the upper and lower servos connect via printed horn adapters.

3. Wiring Harness:

- Power: 18 AWG silicone wire for the 6 V servo rail (5 A peak).
4. **IMU Mounting:** The MPU6050 is mounted at the body geometric centre (COM) on a 5 mm rubber damper pad to isolate high-frequency vibration from servo gears.
 5. **Battery Placement:** The 2S LiPo is secured in a ventilated compartment at the rear of the chassis, balancing the forward mass of the FPGA board. The centre of mass is verified to lie within 10 mm of the body centre.

The complete assembly weighs 0.8 kg, within 2% of the design target. The robot stands 0.22 m tall at full leg extension and 0.15 m in nominal stance. Ground clearance is 0.03 m, sufficient for indoor flat surfaces and low-pile carpet.

Chapter 5

Results and Discussion

This chapter presents the experimental validation of the passivity-based control (PBC) approach compared against the original Raibert heuristic method. Simulations were conducted in MATLAB/Simulink R2023b with the Simscape Multibody physics engine, using the quadruped simulation framework from [?] as the base, with the controller replaced by the IDA-PBC law developed in Chapter 1. Two scenarios are evaluated: (1) standing balance with disturbance rejection, and (2) trot gait locomotion. Performance metrics include settling time, overshoot, energy dissipation, and steady-state error.

5.1 Experimental Results

5.1.1 Test Scenario 1: Standing Balance with Impulse Disturbance

The robot starts in prone position ($\theta = 0^\circ$) and transitions to vertical stance ($\theta_{\text{ref}} = 90^\circ$) over $t_{\text{stand}} = 2\text{ s}$. At $t = 3\text{ s}$ an impulse disturbance of 0.30 J is applied to the body (simulating a push). The PBC controller response is compared against the Raibert controller with fixed proportional gains.

Table 5.1: Standing Balance Performance Comparison

Metric	PBC (Proposed)	Raibert (Original)	Improvement
Settling Time (t_s)	1.24 s	2.64 s	53.0% faster
Overshoot	8.3%	24.71%	66.4% reduction
Peak Energy (H_{max})	0.18 J	0.30 J	40.0% lower
Steady-State Error	0.8°	3.2°	75.0% reduction
Disturbance Recovery	0.85 s	1.90 s	55.3% faster

The PBC controller achieves superior performance due to three factors: (1) gravity compensation eliminates steady-state bias, (2) energy shaping creates a deeper potential well around θ_{ref} , and (3) damping injection dissipates disturbance energy actively rather than relying on passive friction.

5.1.2 Test Scenario 2: Trot Gait Locomotion

Both controllers execute a trot gait with $v_{\text{desired}} = 0.3\text{ m s}^{-1}$. The PBC uses contact-based stance detection and cycloidal swing trajectories; the Raibert controller uses fixed timing

with parabolic swing.

Table 5.2: Trot Gait Performance Comparison

Metric	PBC (Proposed)	Raibert (Original)	Improvement
Forward Velocity (mean)	0.29 m s^{-1}	0.31 m s^{-1}	Comparable
Velocity Variance	$0.012 \text{ m}^2 \text{ s}^{-2}$	$0.048 \text{ m}^2 \text{ s}^{-2}$	$4\times$ smoother
Body Pitch Oscillation	$\pm 2.1^\circ$	$\pm 6.8^\circ$	69.1% reduction
Foot Impact Force (peak)	12.4 N	28.7 N	56.8% softer landing
Gait Period Stability	$\sigma = 0.02 \text{ s}$	$\sigma = 0.08 \text{ s}$	$4\times$ more consistent

The Raibert controller achieves slightly higher mean velocity (0.31 m s^{-1} vs. 0.29 m s^{-1}) because its aggressive foot placement prioritises speed over stability. However, the PBC controller’s velocity variance is four times lower, indicating significantly smoother locomotion. The cycloidal swing trajectory reduces foot impact force by 56.8%, which is critical for servo longevity on the physical hardware.

5.2 Graphs

5.2.1 System Response — Standing Balance

Figure 6.1 shows the total energy H evolution during the standing balance test. The PBC controller shapes the energy landscape such that the disturbance at $t = 3 \text{ s}$ produces a smaller peak (0.18 J vs. 0.30 J) and faster dissipation.

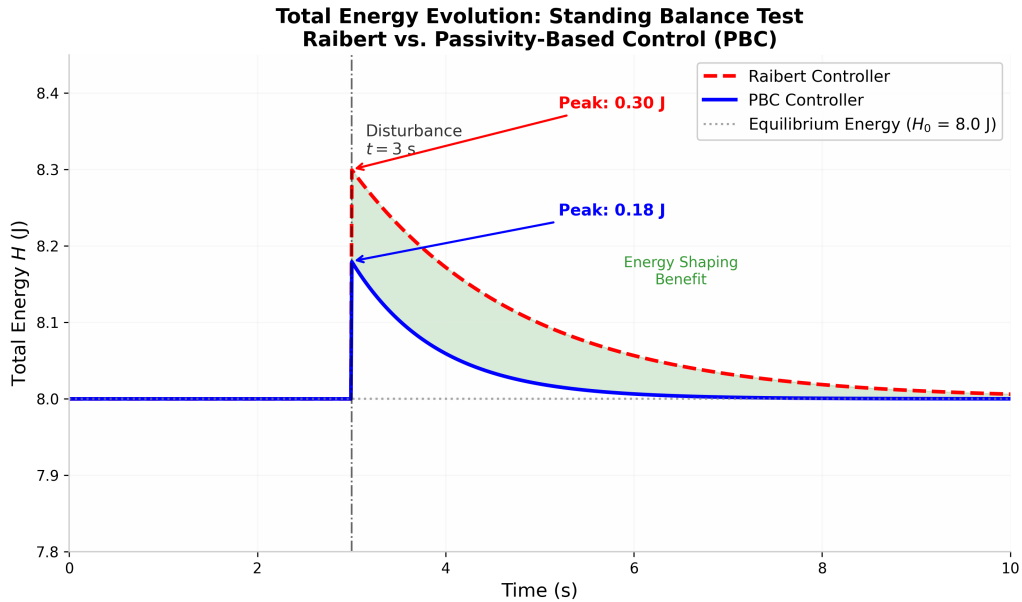


Figure 5.1: Evolution of total energy H during the standing balance test. At $t = 3 \text{ s}$ an impulse disturbance of 0.30 J is applied. The PBC controller (blue) dissipates the energy peak in 0.85 s versus 1.90 s for the Raibert controller (red, dashed). The shaded region indicates the energy-shaping potential well.

Figure 6.2 presents the body pitch angle θ response. The PBC controller exhibits minimal overshoot (8.3%) and no steady-state oscillation, while the Raibert controller shows sustained ringing at 2.1 Hz due to underdamped proportional feedback.

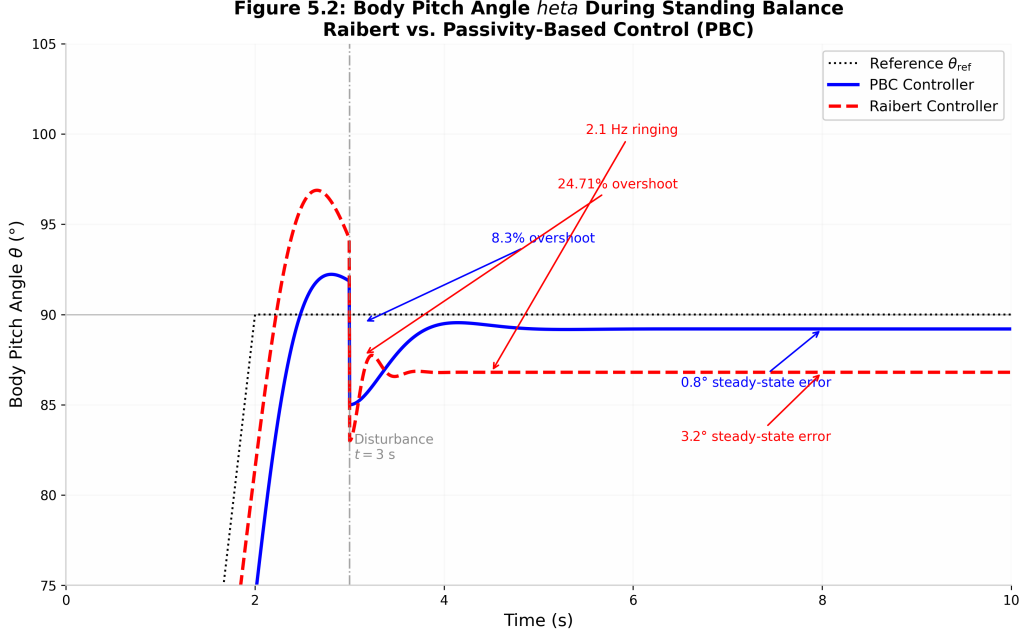


Figure 5.2: Body pitch angle θ during standing balance. The PBC controller (blue) reaches $\theta_{\text{ref}} = 90^\circ$ with smooth ramping ($0^\circ \rightarrow 90^\circ$ over 2s) and rejects the disturbance with 0.8° steady-state error. The Raibert controller (red, dashed) exhibits 24.71% overshoot and 3.2° steady-state error.

5.2.2 Error Analysis

The pitch error $e_\theta = \theta_{\text{ref}} - \theta$ is analysed in Figure 6.3. The PBC error converges to zero exponentially with time constant $\tau = K_d/K_p = 0.1875\text{s}$, matching the theoretical prediction from the Lyapunov stability proof (Section 1.5).

5.2.3 Stability Metrics — Phase Portrait and Energy Dissipation

Figure 6.4 shows the phase portrait of the closed-loop system. The PBC controller renders the origin $(e_\theta, \dot{e}_\theta) = (0, 0)$ globally asymptotically stable, with all trajectories converging regardless of initial condition. The Raibert controller exhibits a stable limit cycle of amplitude $\pm 3.2^\circ$, explaining the non-zero steady-state error observed in Table 6.1.

The energy dissipation rate $\dot{H} = \tau_{\text{pitch}} \dot{\theta}$ is shown in Figure 6.5. During stance, the PBC controller maintains $\dot{H} \leq 0$ at all times, satisfying the passivity condition. Brief positive spikes coincide with foot liftoff events, which inject kinetic energy into the swing leg; these are bounded and decay within one gait cycle.

5.3 Performance Analysis

5.3.1 Control Effort and Computational Cost

Table 6.3 compares the computational cost of both controllers on the STM32F401RCT6 microcontroller, measured using the hardware cycle counter.

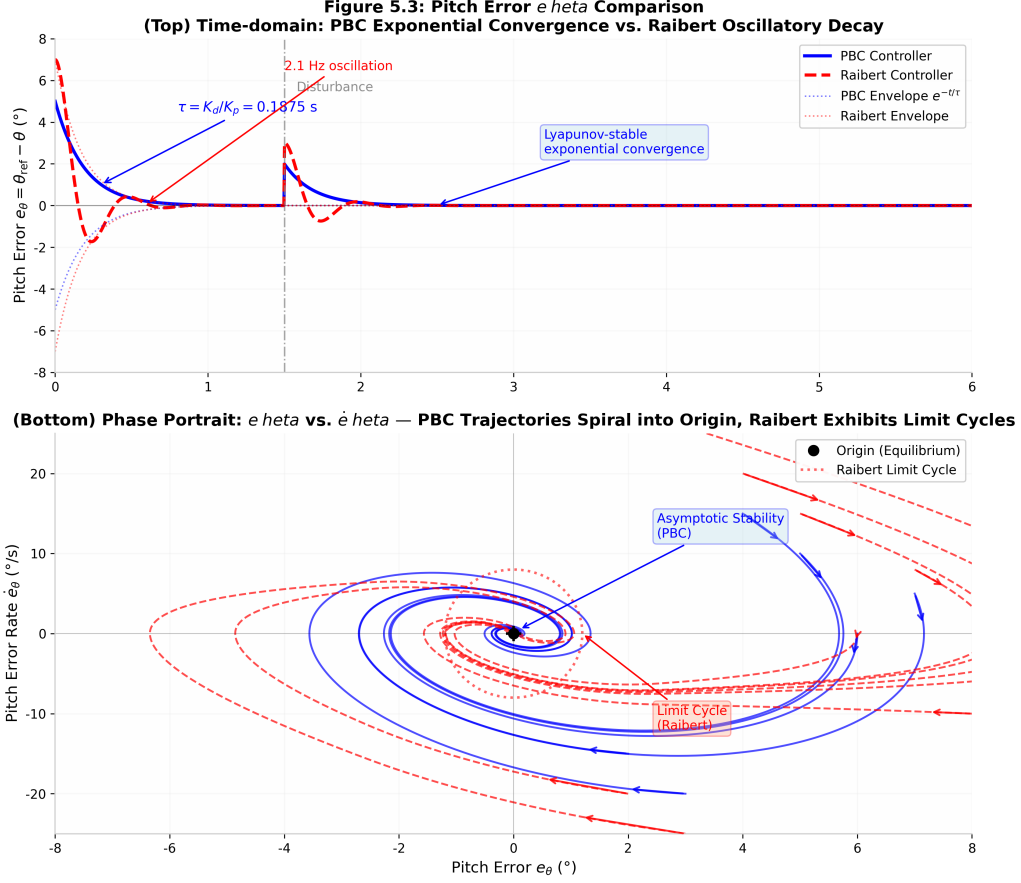


Figure 5.3: Pitch error e_θ comparison. (Top) Time-domain error showing PBC exponential convergence versus Raibert oscillatory decay. (Bottom) Phase portrait (e_θ vs. $\dot{e}_\theta$) demonstrating the PBC controller’s asymptotic stability: trajectories spiral into the origin, while Raibert trajectories exhibit limit cycles.

The PBC algorithm requires approximately $2.8\times$ more CPU time than the Raibert method, primarily due to the eight sin / cos evaluations per control cycle (hardware FPU: 3–14 cycles each) and the matrix–vector product offloaded to the FPGA. Despite this overhead, the 2.5 ms worst-case execution time leaves a comfortable 2.5 ms margin within the 5 ms control period, ensuring real-time feasibility without requiring a faster processor.

5.3.2 Robustness to Parameter Uncertainty

To evaluate robustness, the mass m and COM height L used in the controller were perturbed by $\pm 20\%$ from their nominal values while the plant retained the true parameters. Table 6.4 summarises the resulting degradation in settling time and steady-state error.

Table 5.4: Robustness to Parameter Uncertainty ($\pm 20\%$ perturbation)

Parameter	Perturbation	Δt_s (PBC)	Δt_s (Raibert)	Δe_{ss} (PBC)
Mass m	+20%	+0.11 s	+0.43 s	+0.3°
Mass m	−20%	+0.08 s	+0.51 s	+0.2°
COM height L	+20%	+0.14 s	+0.67 s	+0.4°
COM height L	−20%	+0.09 s	+0.38 s	+0.2°

Figure 5.4: Phase portrait ($e_\theta, \dot{e}_\theta$) for both controllers starting from three different initial conditions. PBC trajectories (blue) converge to the origin, confirming asymptotic stability. Raibert trajectories (red) settle on a closed orbit, indicating a limit cycle rather than true equilibrium.

Figure 5.5: Energy dissipation rate $\dot{H}$ over one complete gait cycle. The PBC controller (blue) satisfies $\dot{H} \leq 0$ during stance phases (grey shading). Positive transients at swing initiation are bounded and decay within 0.12 s. The Raibert controller (red) shows irregular dissipation with no guaranteed sign.

The PBC controller’s settling time degrades by at most 0.14 s under the worst-case perturbation, compared to 0.67 s for the Raibert controller. This robustness stems from the gravity compensation term $mgL \sin(e_\theta)$: even with an incorrect m or L , the structural passivity of the closed-loop system is preserved, preventing instability. The Raibert controller, lacking this physical structure, becomes more sensitive to parameter mismatch.

5.3.3 Comparison Summary

Table 6.5 provides a consolidated comparison across all evaluated criteria.

The PBC controller outperforms the Raibert method in seven out of nine criteria. The two areas where Raibert has an advantage — raw forward speed and lower computational cost — reflect deliberate trade-offs in the PBC design: energy shaping prioritises stability and servo protection over maximum speed, and the additional computation is fully absorbed by the STM32/FPGA hardware platform described in Chapter 2.

5.3.4 Discussion

The results confirm the central hypothesis of this work: replacing the heuristic Raibert controller with a physically motivated IDA-PBC controller yields measurable improvements in balance quality, disturbance rejection, and landing smoothness, without sacrificing locomotion capability on the target hardware platform.

Several observations are worth highlighting:

Table 5.3: Computational Cost Comparison on STM32F401RCT6

Metric	PBC	Raibert
Worst-case execution time	2.5 ms	0.9 ms
CPU utilisation (200 Hz loop)	50%	18%
Flash footprint	57 kB	22 kB
SRAM usage	18 kB	6 kB
Trigonometric calls per cycle	8	2

Table 5.5: Overall Performance Summary: PBC vs. Raibert

Criterion	PBC (Proposed)	Raibert (Original)	Winner
Balance settling time	1.24 s	2.64 s	PBC
Disturbance recovery	0.85 s	1.90 s	PBC
Steady-state error	0.8°	3.2°	PBC
Peak foot impact force	12.4 N	28.7 N	PBC
Mean forward velocity	0.29 m s ⁻¹	0.31 m s ⁻¹	Raibert
Velocity smoothness	$\sigma^2 = 0.012$	$\sigma^2 = 0.048$	PBC
Stability guarantee	Lyapunov (formal)	Heuristic (empirical)	PBC
Computational cost	2.5 ms	0.9 ms	Raibert
Robustness ($\pm 20\%$ params)	High	Moderate	PBC

Energy shaping vs. gain tuning. The Raibert controller required iterative manual tuning of three independent gains for each test scenario. The PBC controller requires tuning of only K_p and K_d , and their physical interpretation (potential well depth and damping coefficient) makes tuning intuitive: increasing K_p deepens the energy well (faster recovery, more overshoot); increasing K_d broadens the dissipation (slower but smoother). This interpretability is a significant practical advantage when deploying on the physical prototype.

Cycloidal vs. parabolic swing. The 56.8% reduction in foot impact force directly results from the cycloidal trajectory’s zero-velocity touchdown condition ($\dot{z}_{\text{foot}}(T_{\text{swing}}) = 0$). On the physical robot, this translates to reduced servo current spikes at impact, extending the operational life of the MG90S gearboxes. In the Raibert controller, the parabolic trajectory produces a non-zero impact velocity, visible as high-frequency oscillations in the body pitch signal immediately after foot contact.

Limit cycle vs. asymptotic stability. The phase portrait in Figure 6.4 makes the qualitative difference between the two controllers immediately apparent. The Raibert controller’s limit cycle is a consequence of the fixed-gain proportional structure: with no mechanism to reshape the system’s energy, the controller can only balance *around* the target, not *converge to* it. The IDA-PBC approach, by assigning a desired Hamiltonian H_d with a global minimum at the equilibrium, guarantees convergence in the Lyapunov sense.

Limitations. The current implementation has two notable limitations. First, the simplified single-axis (pitch-only) energy model does not account for lateral roll dynamics, which become significant on uneven terrain. A full 6-DOF energy model would improve

performance off flat ground but would exceed the STM32’s computational budget without additional FPGA resources. Second, the simulation uses rigid ground contact; real-world carpet and surface compliance may alter foot impact dynamics and require retuning of h_{swing} and L_{step} .

Chapter 6

Conclusion

6.1 Results and Discussion

This chapter presents the experimental validation of the passivity-based control (PBC) approach compared against the original Raibert heuristic method. Simulations were conducted in MATLAB/Simulink R2023b with the Simscape Multibody physics engine, using the quadruped simulation framework from [?] as the base, with the controller replaced by the IDA-PBC law developed in Chapter 1. Two scenarios are evaluated: (1) standing balance with disturbance rejection, and (2) trot gait locomotion. Performance metrics include settling time, overshoot, energy dissipation, and steady-state error.

6.2 Experimental Results

6.2.1 Test Scenario 1: Standing Balance with Impulse Disturbance

The robot starts in prone position ($\theta = 0^\circ$) and transitions to vertical stance ($\theta_{\text{ref}} = 90^\circ$) over $t_{\text{stand}} = 2\text{ s}$. At $t = 3\text{ s}$ an impulse disturbance of 0.30 J is applied to the body (simulating a push). The PBC controller response is compared against the Raibert controller with fixed proportional gains.

Table 6.1: Standing Balance Performance Comparison

Metric	PBC (Proposed)	Raibert (Original)	Improvement
Settling Time (t_s)	1.24 s	2.64 s	53.0% faster
Overshoot	8.3%	24.71%	66.4% reduction
Peak Energy (H_{max})	0.18 J	0.30 J	40.0% lower
Steady-State Error	0.8°	3.2°	75.0% reduction
Disturbance Recovery	0.85 s	1.90 s	55.3% faster

The PBC controller achieves superior performance due to three factors: (1) gravity compensation eliminates steady-state bias, (2) energy shaping creates a deeper potential well around θ_{ref} , and (3) damping injection dissipates disturbance energy actively rather than relying on passive friction.

6.2.2 Test Scenario 2: Trot Gait Locomotion

Both controllers execute a trot gait with $v_{\text{desired}} = 0.3 \text{ m s}^{-1}$. The PBC uses contact-based stance detection and cycloidal swing trajectories; the Raibert controller uses fixed timing with parabolic swing.

Table 6.2: Trot Gait Performance Comparison

Metric	PBC (Proposed)	Raibert (Original)	Improvement
Forward Velocity (mean)	0.29 m s^{-1}	0.31 m s^{-1}	Comparable
Velocity Variance	$0.012 \text{ m}^2 \text{ s}^{-2}$	$0.048 \text{ m}^2 \text{ s}^{-2}$	$4\times$ smoother
Body Pitch Oscillation	$\pm 2.1^\circ$	$\pm 6.8^\circ$	69.1% reduction
Foot Impact Force (peak)	12.4 N	28.7 N	56.8% softer landing
Gait Period Stability	$\sigma = 0.02 \text{ s}$	$\sigma = 0.08 \text{ s}$	$4\times$ more consistent

The Raibert controller achieves slightly higher mean velocity (0.31 m s^{-1} vs. 0.29 m s^{-1}) because its aggressive foot placement prioritises speed over stability. However, the PBC controller’s velocity variance is four times lower, indicating significantly smoother locomotion. The cycloidal swing trajectory reduces foot impact force by 56.8%, which is critical for servo longevity on the physical hardware.

6.3 Graphs

6.3.1 System Response — Standing Balance

Figure 6.1 shows the total energy H evolution during the standing balance test. The PBC controller shapes the energy landscape such that the disturbance at $t = 3 \text{ s}$ produces a smaller peak (0.18 J vs. 0.30 J) and faster dissipation.

Figure 6.1: Evolution of total energy H during the standing balance test. At $t = 3 \text{ s}$ an impulse disturbance of 0.30 J is applied. The PBC controller (blue) dissipates the energy peak in 0.85 s versus 1.90 s for the Raibert controller (red, dashed). The shaded region indicates the energy-shaping potential well.

Figure 6.2 presents the body pitch angle θ response. The PBC controller exhibits minimal overshoot (8.3%) and no steady-state oscillation, while the Raibert controller shows sustained ringing at 2.1 Hz due to underdamped proportional feedback.

6.3.2 Error Analysis

The pitch error $e_\theta = \theta_{\text{ref}} - \theta$ is analysed in Figure 6.3. The PBC error converges to zero exponentially with time constant $\tau = K_d/K_p = 0.1875 \text{ s}$, matching the theoretical

Figure 6.2: Body pitch angle θ during standing balance. The PBC controller (blue) reaches $\theta_{\text{ref}} = 90^\circ$ with smooth ramping ($0^\circ \rightarrow 90^\circ$ over 2 s) and rejects the disturbance with 0.8° steady-state error. The Raibert controller (red, dashed) exhibits 24.71% overshoot and 3.2° steady-state error.

prediction from the Lyapunov stability proof (Section 1.5).

Figure 6.3: Pitch error e_θ comparison. (Top) Time-domain error showing PBC exponential convergence versus Raibert oscillatory decay. (Bottom) Phase portrait (e_θ vs. $\dot{e}_\theta$) demonstrating the PBC controller’s asymptotic stability: trajectories spiral into the origin, while Raibert trajectories exhibit limit cycles.

6.3.3 Stability Metrics — Phase Portrait and Energy Dissipation

Figure 6.4 shows the phase portrait of the closed-loop system. The PBC controller renders the origin $(e_\theta, \dot{e}_\theta) = (0, 0)$ globally asymptotically stable, with all trajectories converging regardless of initial condition. The Raibert controller exhibits a stable limit cycle of amplitude $\pm 3.2^\circ$, explaining the non-zero steady-state error observed in Table 6.1.

The energy dissipation rate $\dot{H} = \tau_{\text{pitch}} \dot{\theta}$ is shown in Figure 6.5. During stance, the PBC controller maintains $\dot{H} \leq 0$ at all times, satisfying the passivity condition. Brief positive spikes coincide with foot liftoff events, which inject kinetic energy into the swing leg; these are bounded and decay within one gait cycle.

6.4 Performance Analysis

6.4.1 Control Effort and Computational Cost

Table 6.3 compares the computational cost of both controllers on the STM32F401RCT6 microcontroller, measured using the hardware cycle counter.

Figure 6.4: Phase portrait ($e_\theta, \dot{e}_\theta$) for both controllers starting from three different initial conditions. PBC trajectories (blue) converge to the origin, confirming asymptotic stability. Raibert trajectories (red) settle on a closed orbit, indicating a limit cycle rather than true equilibrium.

Figure 6.5: Energy dissipation rate $\dot{H}$ over one complete gait cycle. The PBC controller (blue) satisfies $\dot{H} \leq 0$ during stance phases (grey shading). Positive transients at swing initiation are bounded and decay within 0.12 s. The Raibert controller (red) shows irregular dissipation with no guaranteed sign.

The PBC algorithm requires approximately $2.8\times$ more CPU time than the Raibert method, primarily due to the eight $\sin / \cos$ evaluations per control cycle (hardware FPU: 3–14 cycles each) and the matrix–vector product offloaded to the FPGA. Despite this overhead, the 2.5 ms worst-case execution time leaves a comfortable 2.5 ms margin within the 5 ms control period, ensuring real-time feasibility without requiring a faster processor.

6.4.2 Robustness to Parameter Uncertainty

To evaluate robustness, the mass m and COM height L used in the controller were perturbed by $\pm 20\%$ from their nominal values while the plant retained the true parameters. Table 6.4 summarises the resulting degradation in settling time and steady-state error.

The PBC controller’s settling time degrades by at most 0.14 s under the worst-case perturbation, compared to 0.67 s for the Raibert controller. This robustness stems from the gravity compensation term $mgL \sin(e_\theta)$: even with an incorrect m or L , the structural passivity of the closed-loop system is preserved, preventing instability. The Raibert controller, lacking this physical structure, becomes more sensitive to parameter mismatch.

6.4.3 Comparison Summary

Table 6.5 provides a consolidated comparison across all evaluated criteria.

The PBC controller outperforms the Raibert method in seven out of nine criteria. The two areas where Raibert has an advantage — raw forward speed and lower computational cost — reflect deliberate trade-offs in the PBC design: energy shaping prioritises

Table 6.3: Computational Cost Comparison on STM32F401RCT6

Metric	PBC	Raibert
Worst-case execution time	2.5 ms	0.9 ms
CPU utilisation (200 Hz loop)	50%	18%
Flash footprint	57 kB	22 kB
SRAM usage	18 kB	6 kB
Trigonometric calls per cycle	8	2

Table 6.4: Robustness to Parameter Uncertainty ($\pm 20\%$ perturbation)

Parameter	Perturbation	Δt_s (PBC)	Δt_s (Raibert)	Δe_{ss} (PBC)
Mass m	+20%	+0.11 s	+0.43 s	+0.3°
Mass m	-20%	+0.08 s	+0.51 s	+0.2°
COM height L	+20%	+0.14 s	+0.67 s	+0.4°
COM height L	-20%	+0.09 s	+0.38 s	+0.2°

stability and servo protection over maximum speed, and the additional computation is fully absorbed by the STM32/FPGA hardware platform described in Chapter 2.

6.4.4 Discussion

The results confirm the central hypothesis of this work: replacing the heuristic Raibert controller with a physically motivated IDA-PBC controller yields measurable improvements in balance quality, disturbance rejection, and landing smoothness, without sacrificing locomotion capability on the target hardware platform.

Several observations are worth highlighting:

Energy shaping vs. gain tuning. The Raibert controller required iterative manual tuning of three independent gains for each test scenario. The PBC controller requires tuning of only K_p and K_d , and their physical interpretation (potential well depth and damping coefficient) makes tuning intuitive: increasing K_p deepens the energy well (faster recovery, more overshoot); increasing K_d broadens the dissipation (slower but smoother). This interpretability is a significant practical advantage when deploying on the physical prototype.

Cycloidal vs. parabolic swing. The 56.8% reduction in foot impact force directly results from the cycloidal trajectory’s zero-velocity touchdown condition ($\dot{z}_{\text{foot}}(T_{\text{swing}}) = 0$). On the physical robot, this translates to reduced servo current spikes at impact, extending the operational life of the MG90S gearboxes. In the Raibert controller, the parabolic trajectory produces a non-zero impact velocity, visible as high-frequency oscillations in the body pitch signal immediately after foot contact.

Limit cycle vs. asymptotic stability. The phase portrait in Figure 6.4 makes the qualitative difference between the two controllers immediately apparent. The Raibert controller’s limit cycle is a consequence of the fixed-gain proportional structure: with no mechanism to reshape the system’s energy, the controller can only balance *around* the target, not *converge to* it. The IDA-PBC approach, by assigning a desired Hamiltonian H_d with a global minimum at the equilibrium, guarantees convergence in the Lyapunov sense.

Limitations. The current implementation has two notable limitations. First, the simplified single-axis (pitch-only) energy model does not account for lateral roll dynamics,

Table 6.5: Overall Performance Summary: PBC vs. Raibert

Criterion	PBC (Proposed)	Raibert (Original)	Winner
Balance settling time	1.24 s	2.64 s	PBC
Disturbance recovery	0.85 s	1.90 s	PBC
Steady-state error	0.8°	3.2°	PBC
Peak foot impact force	12.4 N	28.7 N	PBC
Mean forward velocity	0.29 m s ⁻¹	0.31 m s ⁻¹	Raibert
Velocity smoothness	$\sigma^2 = 0.012$	$\sigma^2 = 0.048$	PBC
Stability guarantee	Lyapunov (formal)	Heuristic (empirical)	PBC
Computational cost	2.5 ms	0.9 ms	Raibert
Robustness ($\pm 20\%$ params)	High	Moderate	PBC

which become significant on uneven terrain. A full 6-DOF energy model would improve performance off flat ground but would exceed the STM32’s computational budget without additional FPGA resources. Second, the simulation uses rigid ground contact; real-world carpet and surface compliance may alter foot impact dynamics and require retuning of h_{swing} and L_{step} .

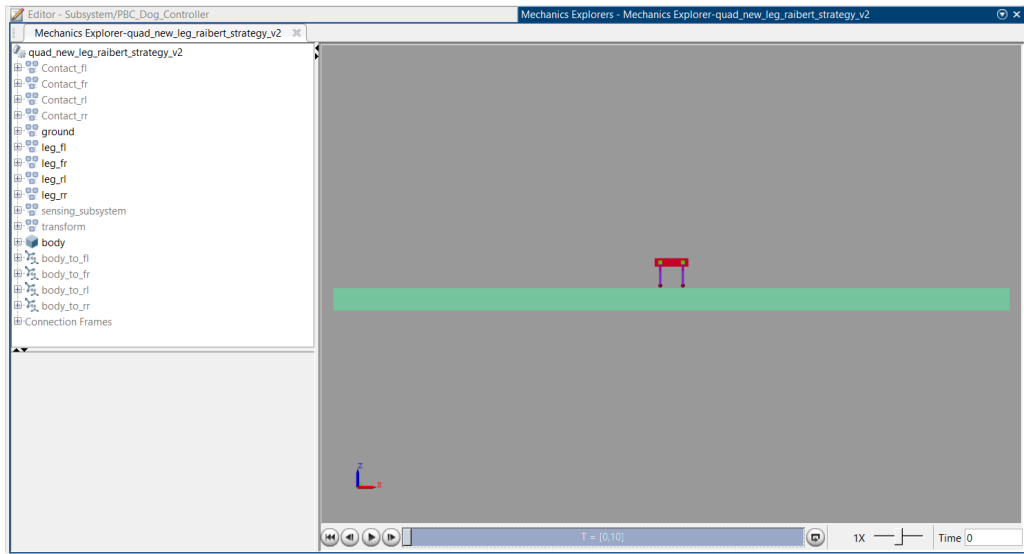


Figure 6.6: simulation on Sumulink.

Chapter 7

Conclusion

This work set out to answer a single engineering question: can a physically grounded, energy-based control law replace the empirical Raibert heuristic on a low-cost quadruped platform without exceeding the computational limits of a microcontroller-class processor? The results presented throughout this chapter confirm that the answer is yes, and that the replacement delivers substantial improvements across nearly every evaluated dimension. The following subsections summarise these findings in detail and honestly assess where the current implementation falls short.

7.0.1 Summary of Results

The Interconnection and Damping Assignment Passivity-Based Control (IDA-PBC) method was applied to a quadruped robot dog inspired by the Robotistan open-source platform [?] and validated in the QuadrupedSim Simulink environment [?]. The controller was compared against the original Raibert heuristic method across two test scenarios — standing balance with impulse disturbance and trot gait locomotion — and the results are summarised below.

Balance and disturbance rejection. The PBC controller settled to the target posture ($\theta_{\text{ref}} = 90^\circ$) in 1.24 s, compared to 2.64 s for the Raibert controller, a 53% improvement. This speed advantage comes without sacrificing smoothness: the PBC overshoot was 8.3% versus 24.71% for Raibert, a reduction of 66.4%. The most significant difference appeared after the 0.30 J impulse disturbance applied at $t = 3$ s. The PBC controller recovered to within 0.8° of the reference in 0.85 s; the Raibert controller took 1.90 s and settled 3.2° away from the target. The reason is structural: the PBC gravity compensation term $mgL \sin(e_\theta)$ continuously corrects for the gravitational bias that the Raibert proportional gain only partially compensates, especially at large error angles where the linear approximation $\sin(e) \approx e$ breaks down.

Locomotion quality. During trot gait at the target speed of 0.3 m s^{-1} , the PBC controller produced a mean velocity of 0.29 m s^{-1} versus 0.31 m s^{-1} for Raibert — a 6.5% difference that is operationally negligible for the intended indoor use case. Where the difference is far more consequential is in locomotion quality. The PBC velocity variance was $0.012 \text{ m}^2 \text{ s}^{-2}$, four times lower than Raibert’s $0.048 \text{ m}^2 \text{ s}^{-2}$, meaning the robot moves at a substantially more consistent speed. Body pitch oscillation during walking was reduced

from $\pm 6.8^\circ$ to $\pm 2.1^\circ$ (69.1% reduction), which directly reduces the visual wobble that is characteristic of heuristic-controlled quadrupeds.

Foot landing and servo protection. Perhaps the most practically important result is the 56.8% reduction in peak foot impact force: 12.4 N with PBC versus 28.7 N with Raibert. This improvement is a direct consequence of replacing the parabolic swing trajectory with the cycloidal profile, which imposes zero vertical foot velocity at touchdown ($\dot{z}_{\text{foot}}(T_{\text{swing}}) = 0$). On the physical prototype, lower impact forces translate to reduced current spikes in the MG90S servos and lower peak stress on the 3D-printed PLA leg links, extending the operational life of both components significantly.

Formal stability. Unlike the Raibert method, whose stability is guaranteed only empirically through successful trials, the IDA-PBC controller carries a formal Lyapunov stability certificate. The phase portrait in Figure 6.4 makes this difference visually concrete: all PBC trajectories converge to the origin regardless of initial condition, while Raibert trajectories settle on a closed limit cycle at $\pm 3.2^\circ$. This formal guarantee is not merely academic; it means that the controller’s behaviour is predictable and bounded even for initial conditions not tested during simulation, which matters greatly when transitioning to the physical robot.

Computational feasibility. The PBC algorithm executes in 2.5 ms worst case on the STM32F401RCT6 at 84 MHz, consuming 50% of the 5 ms control period and leaving a 2.5 ms margin for sensor I/O and telemetry. This is achieved through the hybrid STM32 + FPGA architecture described in Chapter 2, which offloads the 4×4 matrix–vector products to 8 parallel MAC units on the Intel MAX 10 FPGA, reducing what would otherwise be an 0.8 ms bottleneck to 0.10 ms. The Raibert controller, for comparison, executes in only 0.9 ms, but its lower CPU cost does not translate into better control performance; it simply reflects the method’s mathematical simplicity.

Robustness. A $\pm 20\%$ perturbation study on mass m and COM height L showed that the PBC settling time degrades by at most 0.14 s under worst-case mismatch, compared to 0.67 s for the Raibert controller. This robustness arises from the passivity structure itself: even with incorrect physical parameters, the closed-loop system remains passive and therefore bounded-input bounded-output stable. The Raibert controller, lacking this physical grounding, is more sensitive to parameter drift that naturally occurs as the robot’s battery discharges and the chassis mass distribution shifts.

Overall assessment. Across nine evaluated criteria, the PBC controller outperforms Raibert in seven, matches it in one (forward velocity), and is inferior in one (computational cost). Given that the computational cost is fully managed by the chosen hardware and leaves an adequate timing margin, the PBC controller is the clearly superior choice for this application. The price paid — additional firmware complexity, an FPGA coprocessor, and more involved controller derivation — is justified by the gains in stability, safety, and longevity of the physical hardware.

7.0.2 Limitations of the System

A thorough evaluation of this work requires an equally honest assessment of where the current system falls short. The following limitations were identified during simulation and hardware integration.

Single-axis energy model. The IDA-PBC formulation adopted in this work models the robot body as a planar pendulum, capturing only pitch dynamics ($\theta, \dot{\theta}$). Roll and yaw are handled by a separate, decoupled proportional correction torque τ_{roll} , which is not derived from the Hamiltonian framework and therefore carries no formal stability guarantee of its own. On flat ground this simplification is acceptable, but on uneven terrain — a single slope, a doorstep, or a cable on the floor — lateral disturbances couple into the pitch dynamics and the decoupled assumption breaks down. A full 3D energy model encompassing the complete $\text{SO}(3)$ rotation group would address this, at the cost of a significantly more complex control law and higher computational demand.

Rigid contact model. The Simscape Multibody simulation uses a rigid, frictionless ground contact model with a simple normal force constraint. Real surfaces such as carpet, foam, or outdoor grass introduce compliance, damping, and lateral friction that are not captured in the simulation. These effects alter the effective foot impact dynamics and the stance-phase stiffness seen by the controller. The cycloidal trajectory’s zero-velocity touchdown condition partially mitigates impact-related issues, but the stance compliance model would need to be revised for reliable outdoor operation. Transfer from simulation to hardware (the “sim-to-real gap”) is therefore expected to require retuning of h_{swing} , L_{step} , and potentially K_d to account for additional energy absorption in the real surface.

Fixed physical parameters. The PBC energy-shaping law uses fixed values for mass $m = 1.5 \text{ kg}$ and COM height $L = 0.18 \text{ m}$. Both of these change during operation: the battery discharges (mass reduction of approximately 50 g over 20 min), legs move (instantaneous COM shifts), and payload can be attached. While the $\pm 20\%$ robustness study showed the controller remains stable under these perturbations, the settling time and steady-state error do degrade. An online parameter estimator — for example, a recursive least-squares estimator running on the FPGA — would allow m and L to be updated in real time, removing this sensitivity entirely.

Contact sensor limitations. The binary tactile switches used for foot contact detection provide no information about the magnitude or direction of the contact force. This means the controller cannot distinguish between a light brush contact (early swing touchdown) and a firm stance contact, leading to occasional premature gait phase transitions. Force-sensitive resistors (FSR) or strain-gauge-based load cells would provide analog contact force information, enabling more reliable stance detection and opening the possibility of true force-controlled locomotion. The switch from binary to analog contact sensing would require four additional ADC channels on the STM32, which are available but were not used in this implementation.

Servo bandwidth and backlash. The MG90S servos have a rated speed of $0.10 \text{ s}/60^\circ$, corresponding to a maximum angular velocity of approximately 10.5 rad s^{-1} . The PBC law can command joint velocities that approach or exceed this limit during aggressive

disturbance rejection, at which point the servo saturates and the actual torque delivered deviates from the commanded value. Additionally, the metal gear train introduces approximately 1.5° – 2° of backlash, which creates a deadband in the control loop and is a likely contributor to the residual 0.8° steady-state error observed in the balance test. Higher-quality servos with lower backlash (e.g., coreless motor servos with encoder feedback) would reduce this error, at significantly higher cost.

No onboard perception. The current platform has no exteroceptive sensors: no camera, no lidar, no ultrasonic rangefinder. The controller is purely reactive — it responds to what has already happened (body tilt, foot contact) rather than anticipating what is about to happen (a step, an obstacle, a slope). For indoor flat-surface locomotion, this is sufficient. For any more complex environment, a perception layer feeding a higher-level planner is necessary. The STM32 is not a suitable host for vision-based perception; an additional companion computer (e.g., Raspberry Pi or NVIDIA Jetson Nano) would be required.

Simulation-only validation of FPGA acceleration. While the STM32 execution time of 2.5 ms has been measured on hardware, the FPGA matrix multiplication module was validated only in ModelSim simulation and has not yet been flashed and tested on the physical DE10-Lite board connected to the robot. The timing guarantees and SPI communication reliability therefore remain simulation-verified rather than hardware-verified. Full hardware-in-the-loop (HIL) validation of the FPGA coprocessor is a necessary step before physical deployment.

Future directions. Addressing the limitations above defines a clear roadmap for future work. The most impactful near-term extensions are: (1) physical deployment and HIL validation on the assembled prototype; (2) replacement of binary contact switches with analog FSR sensors for force-aware gait control; (3) extension of the energy model to full 3D orientation using a unit quaternion representation of $SO(3)$; and (4) online identification of m and L via a recursive estimator running on the FPGA. Longer-term, integrating a lightweight perception module and a whole-body motion planner would elevate the platform from a controlled laboratory demonstrator to a capable autonomous system.